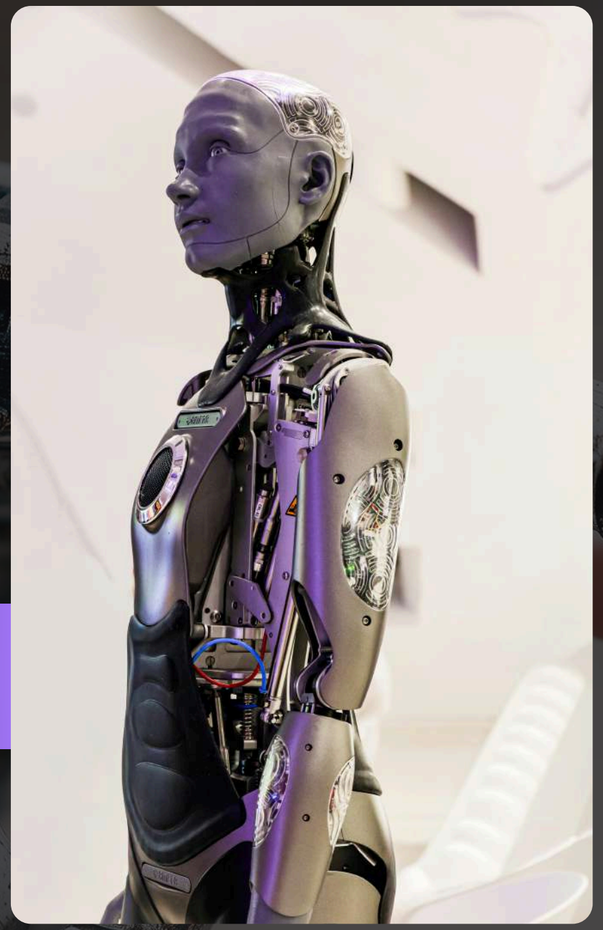


UNIT 4

Deep Learning

Recurrent Neural Network

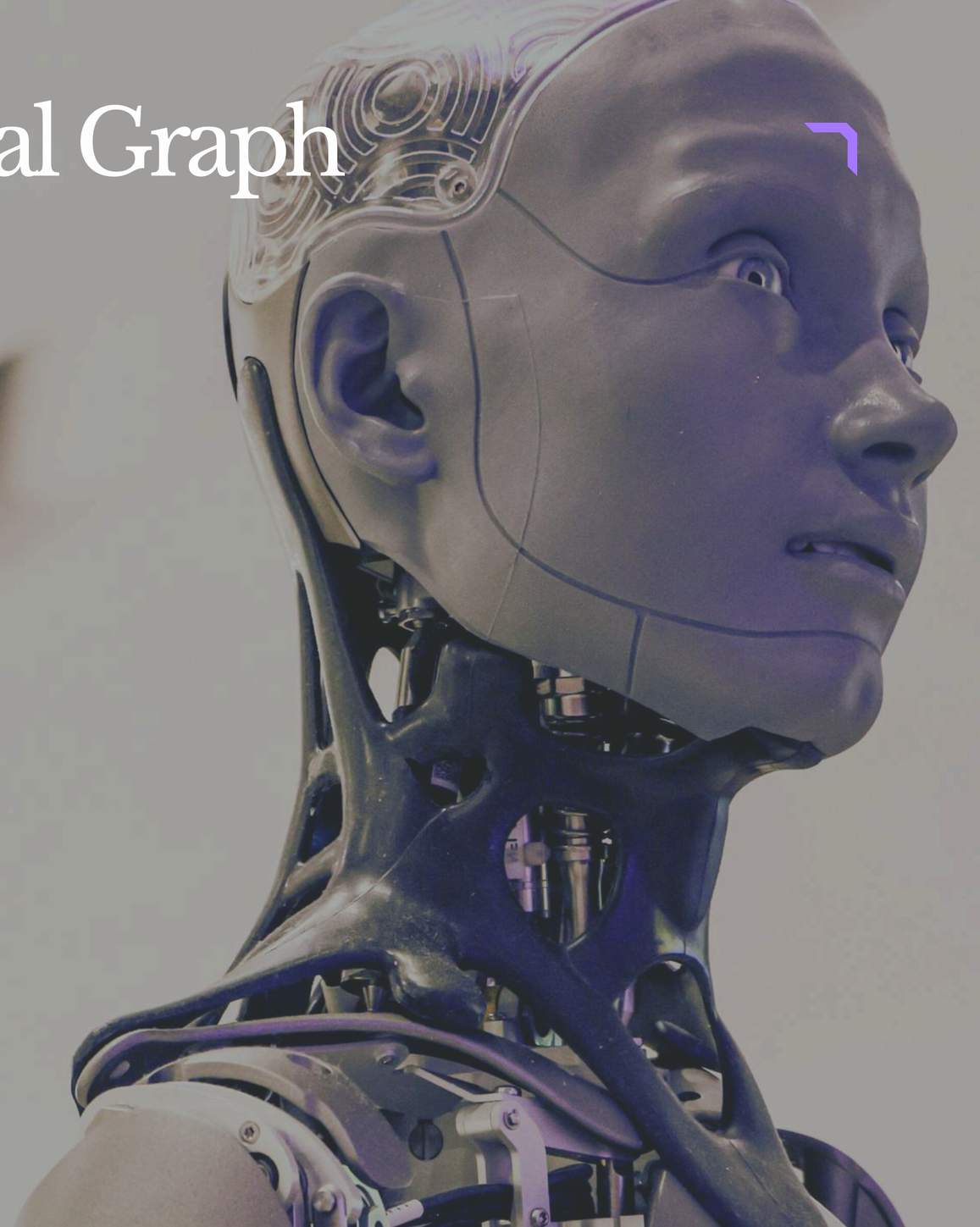
Unfolding Computational Graph

What is a Computational Graph?

- A computational graph is a visual representation of operations performed on inputs to produce an output.
- Nodes represent operations such as:

1. Addition
2. Multiplication
3. Activation functions

- Edges represent the flow of data between operations.
- Used in deep learning models for computation and training.

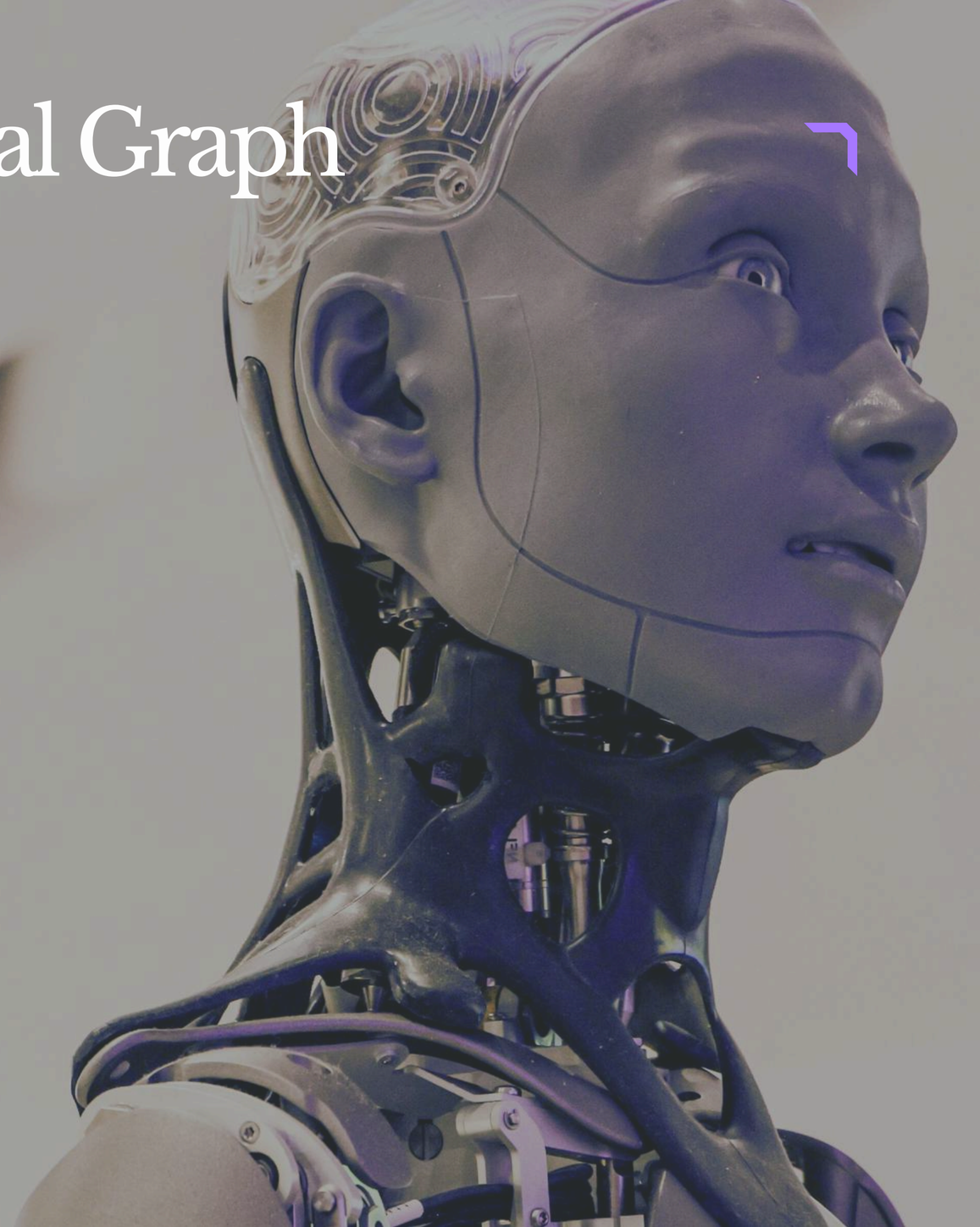


Unfolding Computational Graph

What is Unfolding?

- **Unfolding means expanding repeated operations across multiple time steps.**
- **It converts a loop-like process into a sequence of individual operations.**
- **Also called “unrolling” a computational graph.**
- **Helps visualize how data flows over time.**

Why is Unfolding Important?

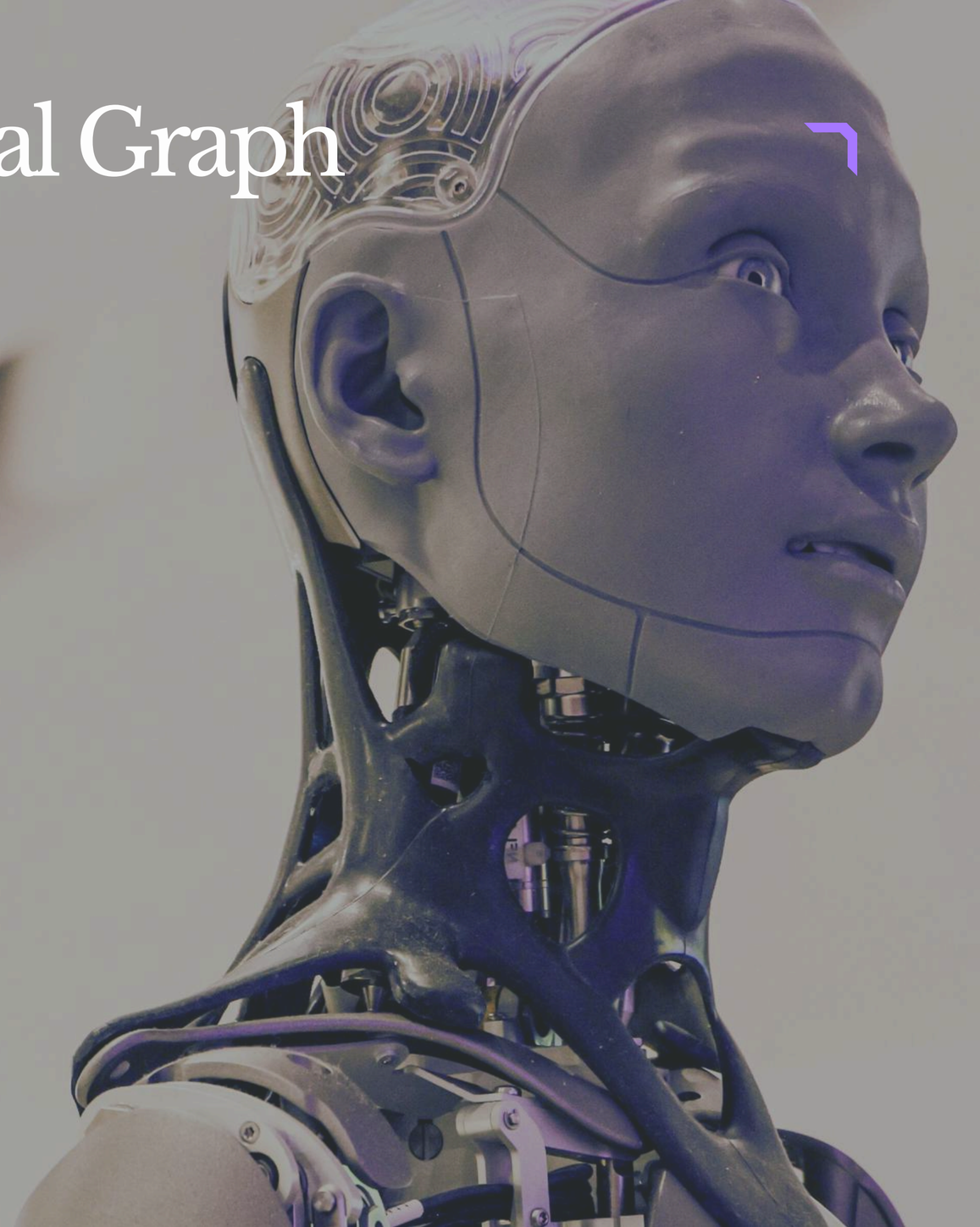


Unfolding Computational Graph

Why is Unfolding Important?

- **Commonly used in Recurrent Neural Networks (RNNs).**
- **Helps understand sequential data processing.**
- **Makes training using Backpropagation Through Time (BPTT) easier.**
- **Shows how hidden states are passed from one step to another.**

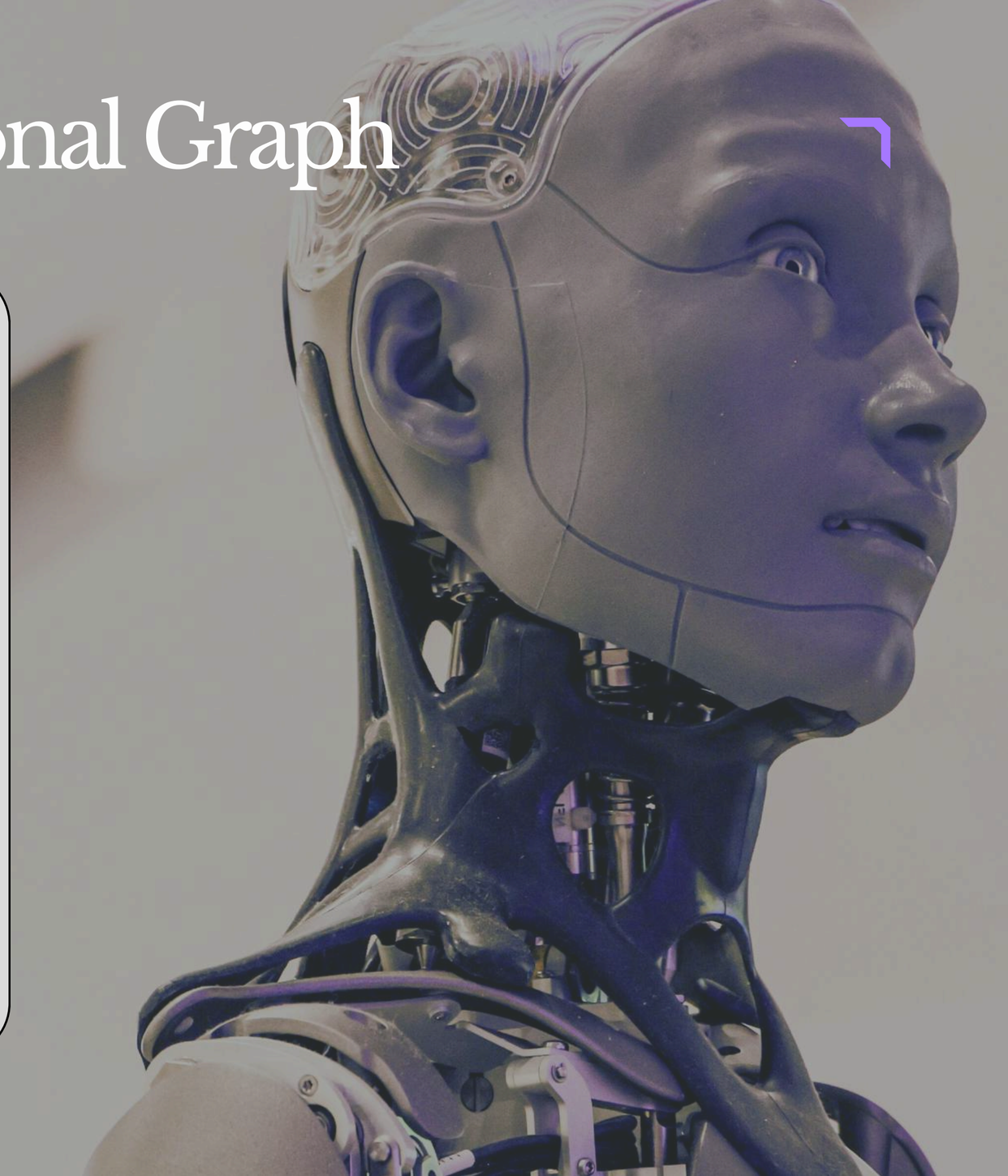
Unfolding in RNN



Unfolding Computational Graph

- In RNN, the same function is repeated at every time step.
- At each time step, the RNN takes:
- Current input xtx_txt
- Previous hidden state ht−1h_{t-1}ht−1
- Produces:
- New hidden state hth_tht
- Output yt
- Inline formula for RNN:

  ht=F(xt,ht−1)h_t

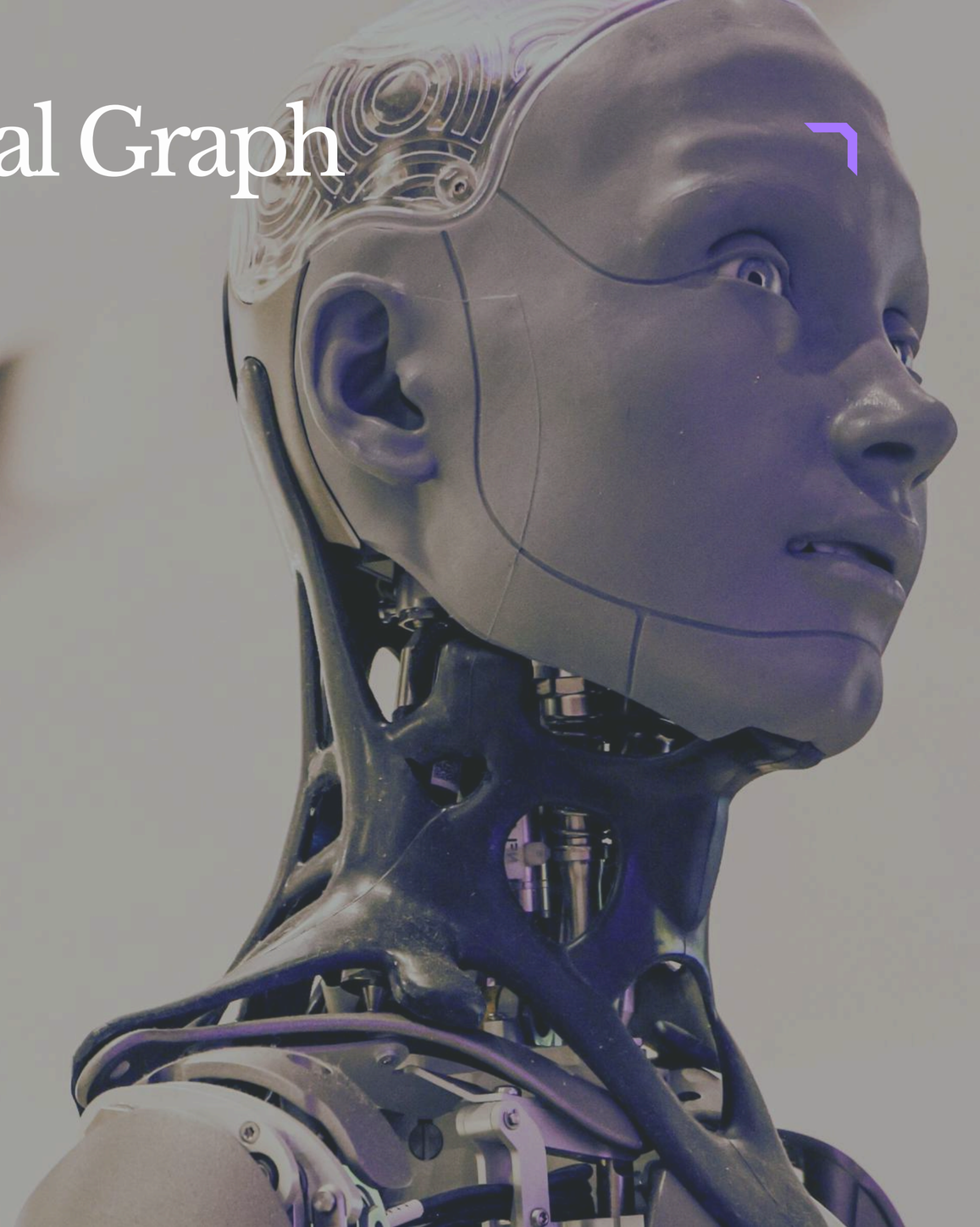


Unfolding Computational Graph

Example of Unfolding

- Consider processing a sentence word by word.
- Each word is processed at a different time step.
- The same RNN function FFF is reused with different inputs.
- After unfolding for multiple time steps:
- Multiple copies of the same function appear.
- Each copy handles one time step.

Advantages



Unfolding Computational Graph

- Easy visualization of sequential processing.
- Helps in gradient calculation during training.
- Useful for handling:
- Text
- Speech
- Time-series data
- Improves understanding of memory flow in RNNs.

Recurrent Neural Network (RNN)

Introduction to RNN

- **Recurrent Neural Network (RNN) is a type of neural network.**
- **Designed to work with sequential data such as:**
- **Text**
- **Audio**
- **Time-series data**
- **Unlike traditional neural networks, RNNs can remember previous inputs.**
- **Uses past information to improve predictions.**

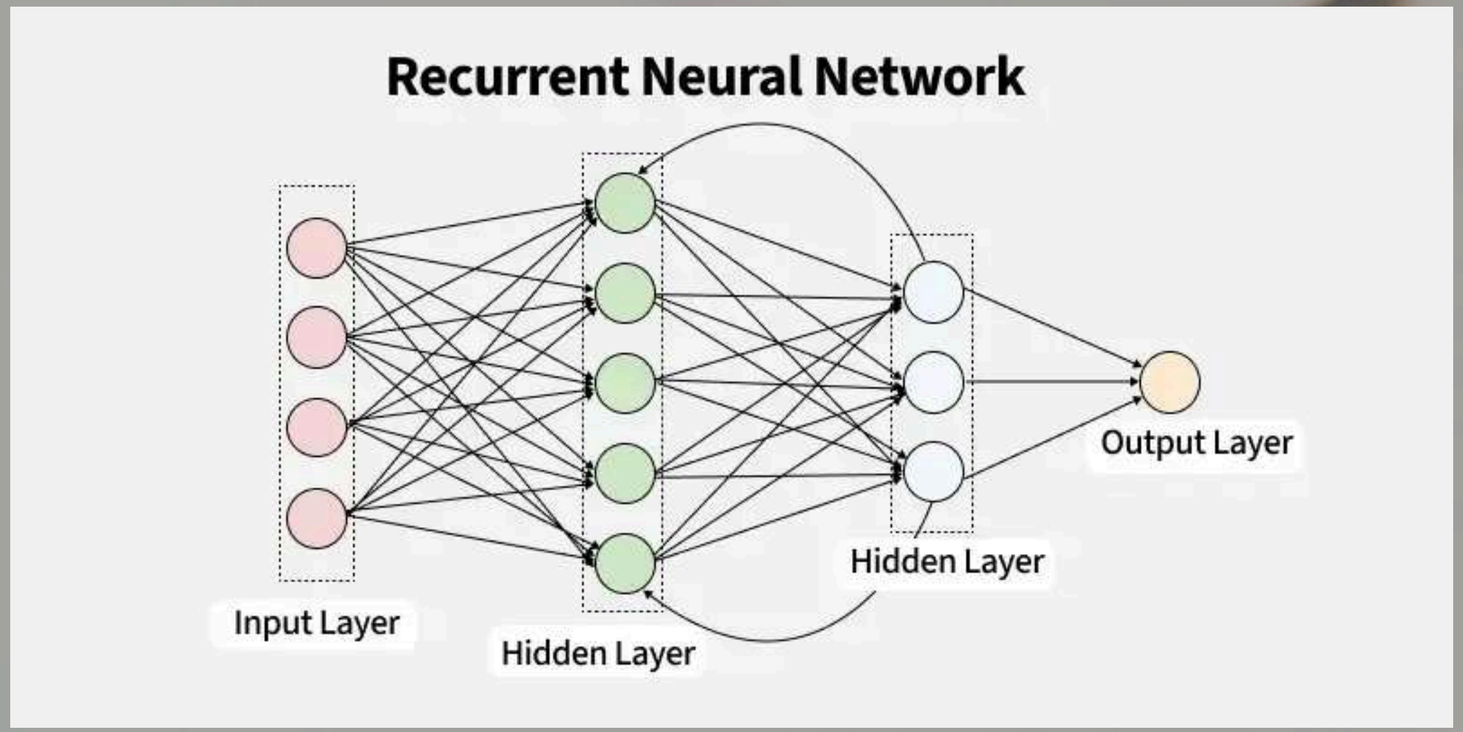
Recurrent Neural Network (RNN)

Key Feature of RNN

- **RNN has memory of previous inputs.**
- **Output from the previous step is fed back into the network.**
- **This feedback helps the network learn patterns over time.**
- **Suitable for sequence-based tasks.**

Architecture of RNN

Recurrent Neural Network (RNN)



Recurrent Neural Network (RNN)

Working of RNN

- RNN processes data one step at a time.
- Information is passed from one step to the next using hidden states.
- At each time step, RNN takes:
- Current input xt
- Previous hidden state ht−1
- Produces:
- New hidden state hth_tht
- Output yt
- RNN hidden state equation:
- ht=F(xt,ht−1)h_t

Recurrent Neural Network (RNN)

Step-by-Step Processing

- Initialize hidden state h0 (usually zero).
- Feed first input x1 into the RNN.
- Compute hidden state h1.
- Feed next input x2along with h1.
- Continue the process for the entire sequence.
- Final output may be generated:
- At every time step
- Or only at the end.

Recurrent Neural Network (RNN)

Need of RNN

- Required for tasks where input order matters.
- Traditional neural networks process inputs independently.
- They cannot remember previous information.
- RNN solves this problem using hidden state memory.
- Past information influences future outputs.

Types of RNN

Introduction

- **RNN architectures can handle different types of input and output sequences.**

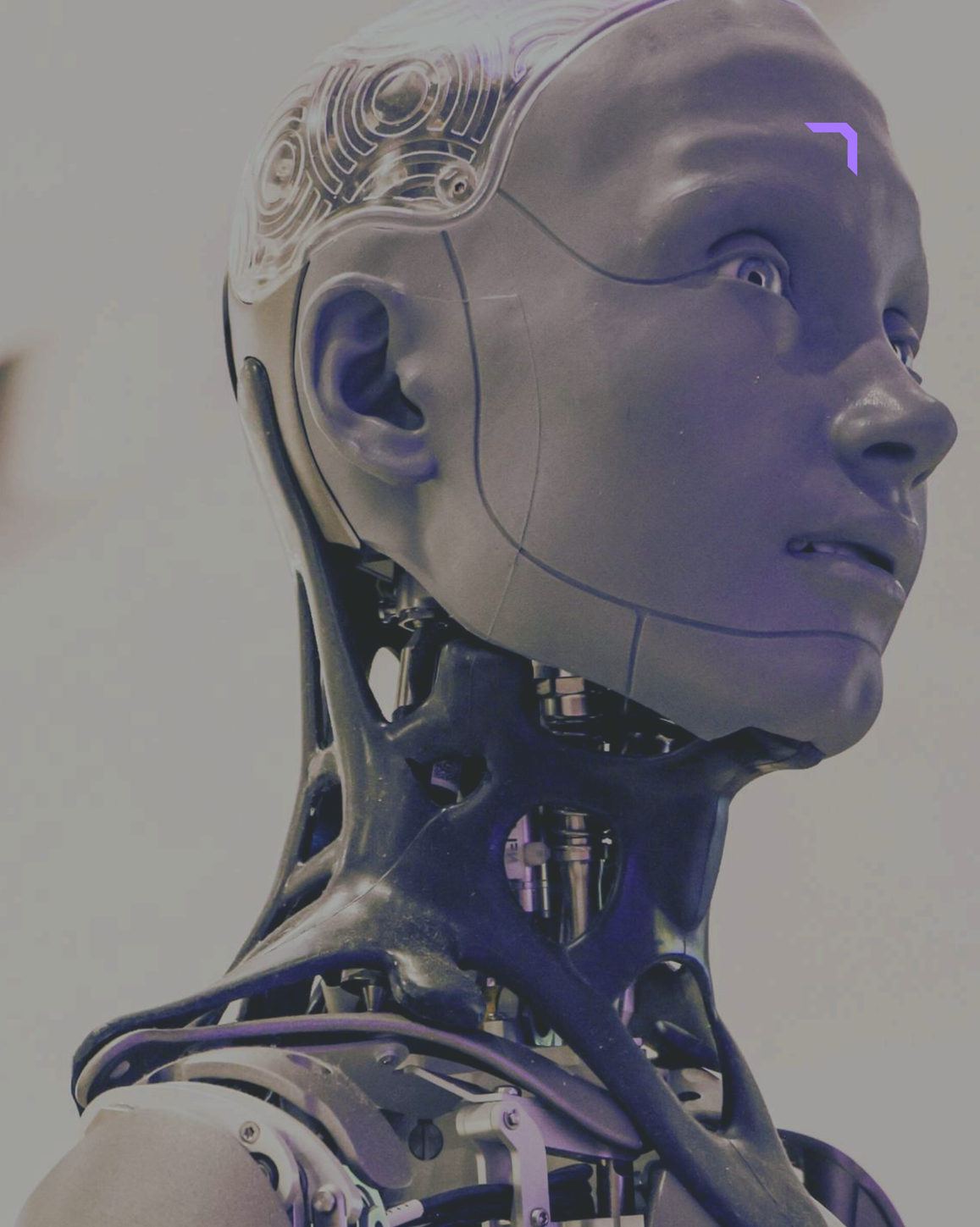
Types depend on:

- **Number of inputs**
- **Number of outputs**

Sequence processing style

Major types:

- **One-to-One**
- **One-to-Many**
- **Many-to-One**
- **Many-to-Many**



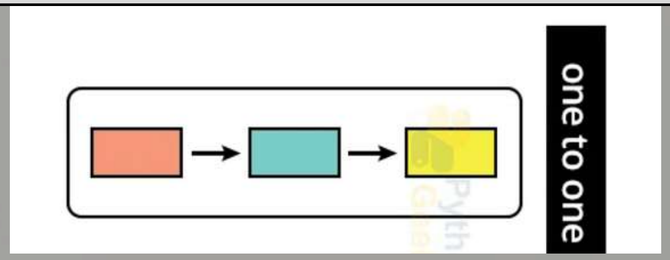
Types of RNN

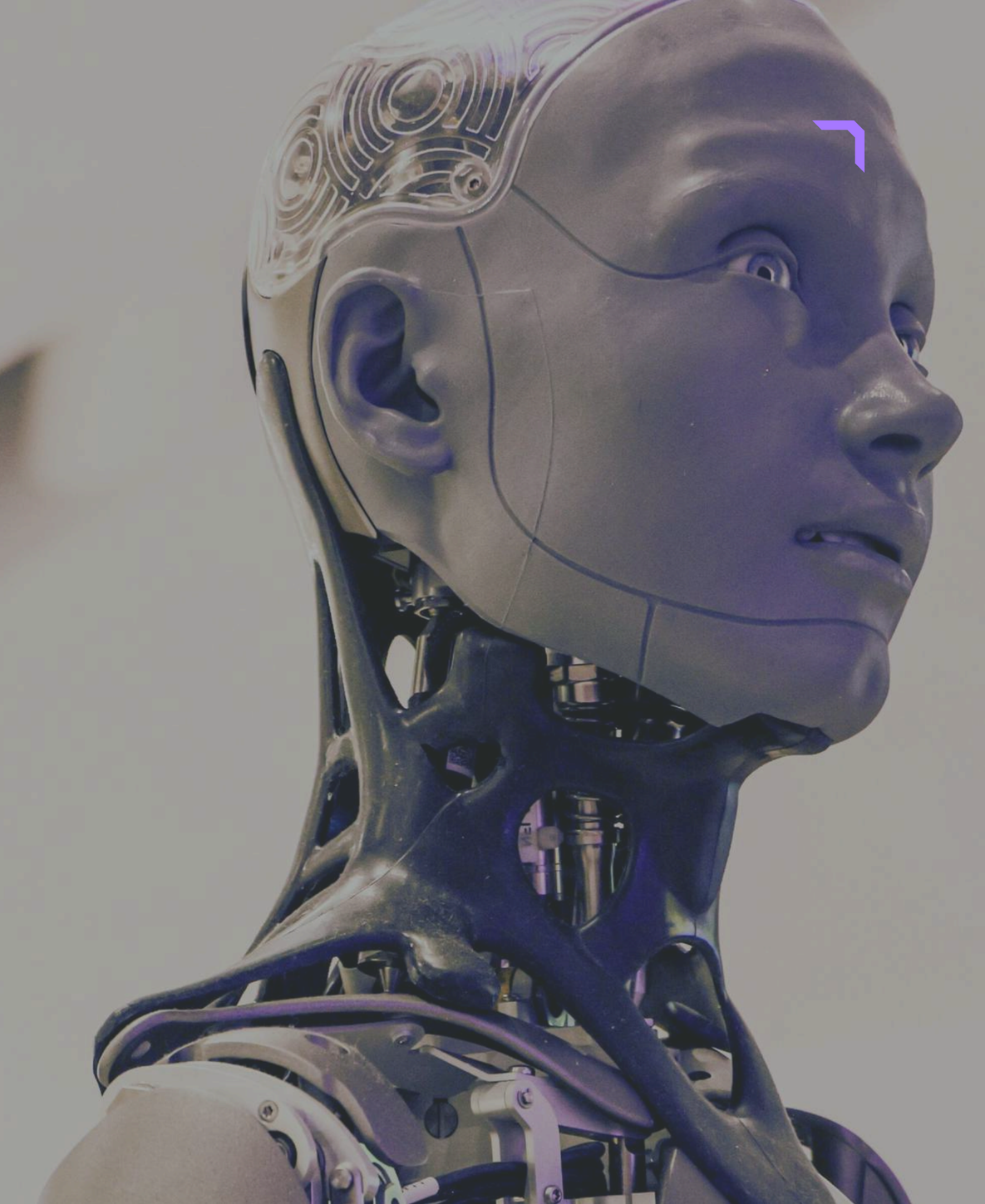
1. One-to-One RNN

- Simplest form of neural network.
- Does not involve sequence data.
- One input produces one output.
- Similar to traditional neural networks.

Example:

- Image classification
- Input: Image
- Output: Label such as “Cat” or “Dog”





Types of RNN

2. One-to-Many RNN

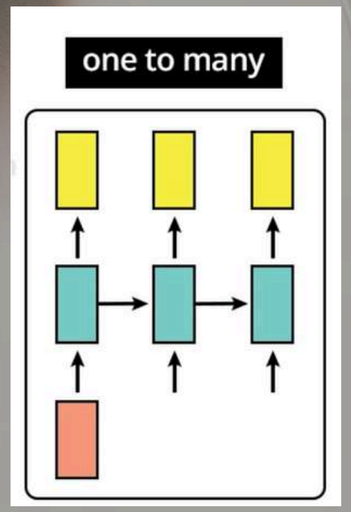
- Single input generates a sequence of outputs.
- RNN remembers previous outputs while generating the next output.

Example:

- Image Captioning
- Input: One image
- Output: Full descriptive sentence

Working:

- The model generates words one by one.
- Uses memory to decide the next word.



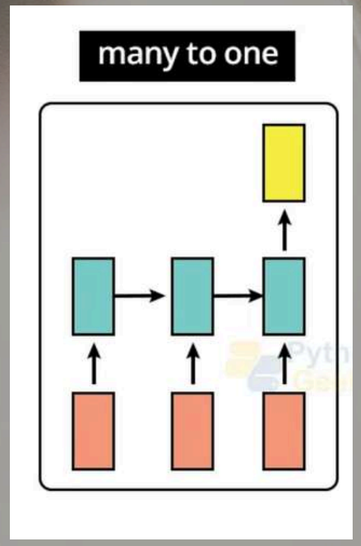
Types of RNN

3. Many-to-One RNN

- A sequence of inputs produces a single output.
- Entire sequence is processed before final prediction.

Example:

- Sentiment Analysis
- Input: Sentence (word-by-word)
- Output:
- Positive
- Negative
- Neutral



Types of RNN

4. Many-to-Many RNN

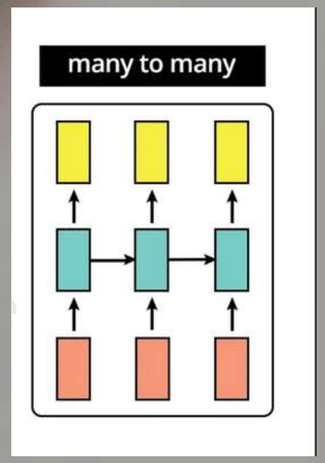
- Both input and output are sequences.
- Input and output sequences may have:
- Same length
- Different lengths

Example:

- Part-of-Speech Tagging
- Each word gets a grammatical label.

Other Examples:

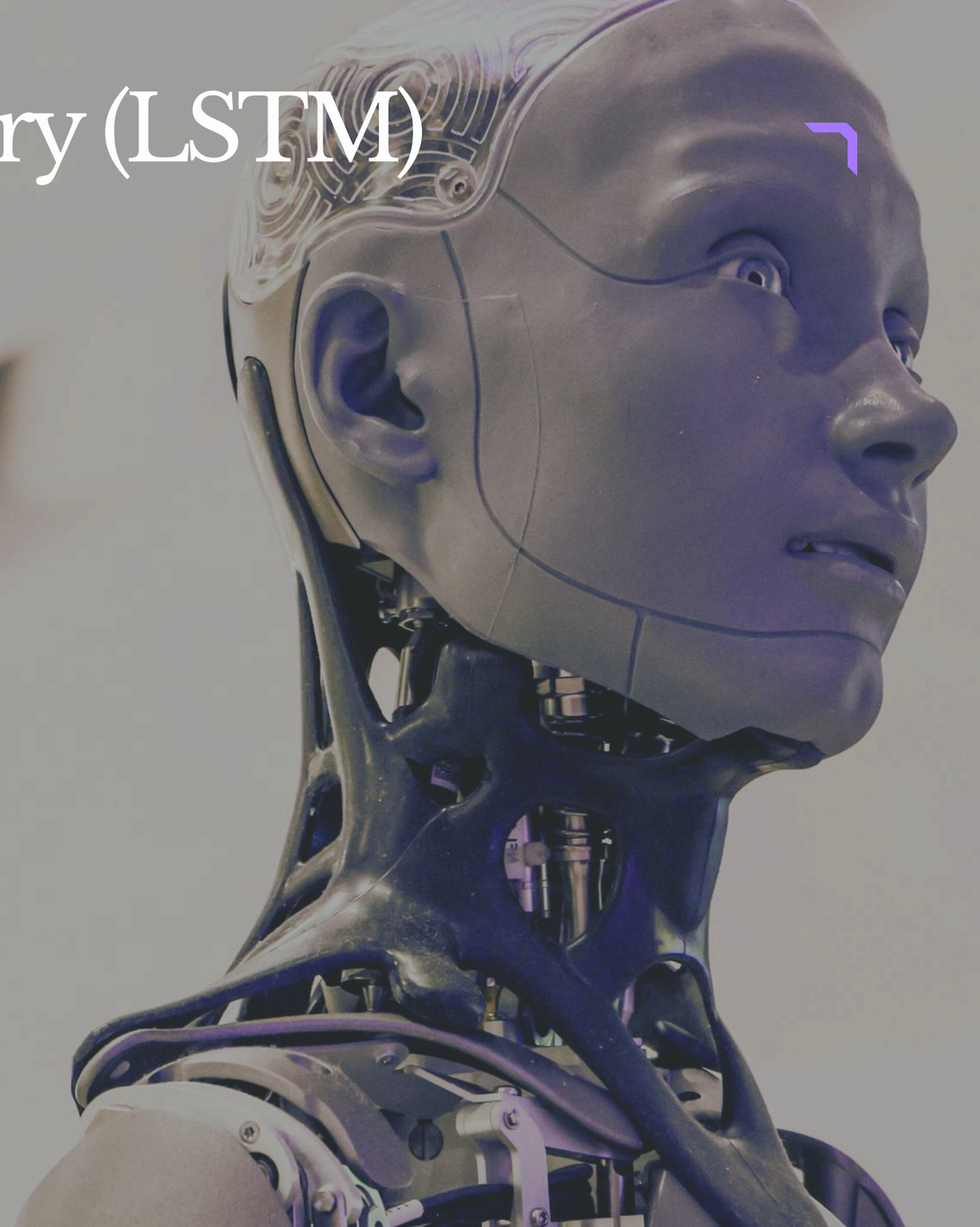
- Machine Translation
- Chatbots



Long Short-Term Memory (LSTM)

Introduction to LSTM

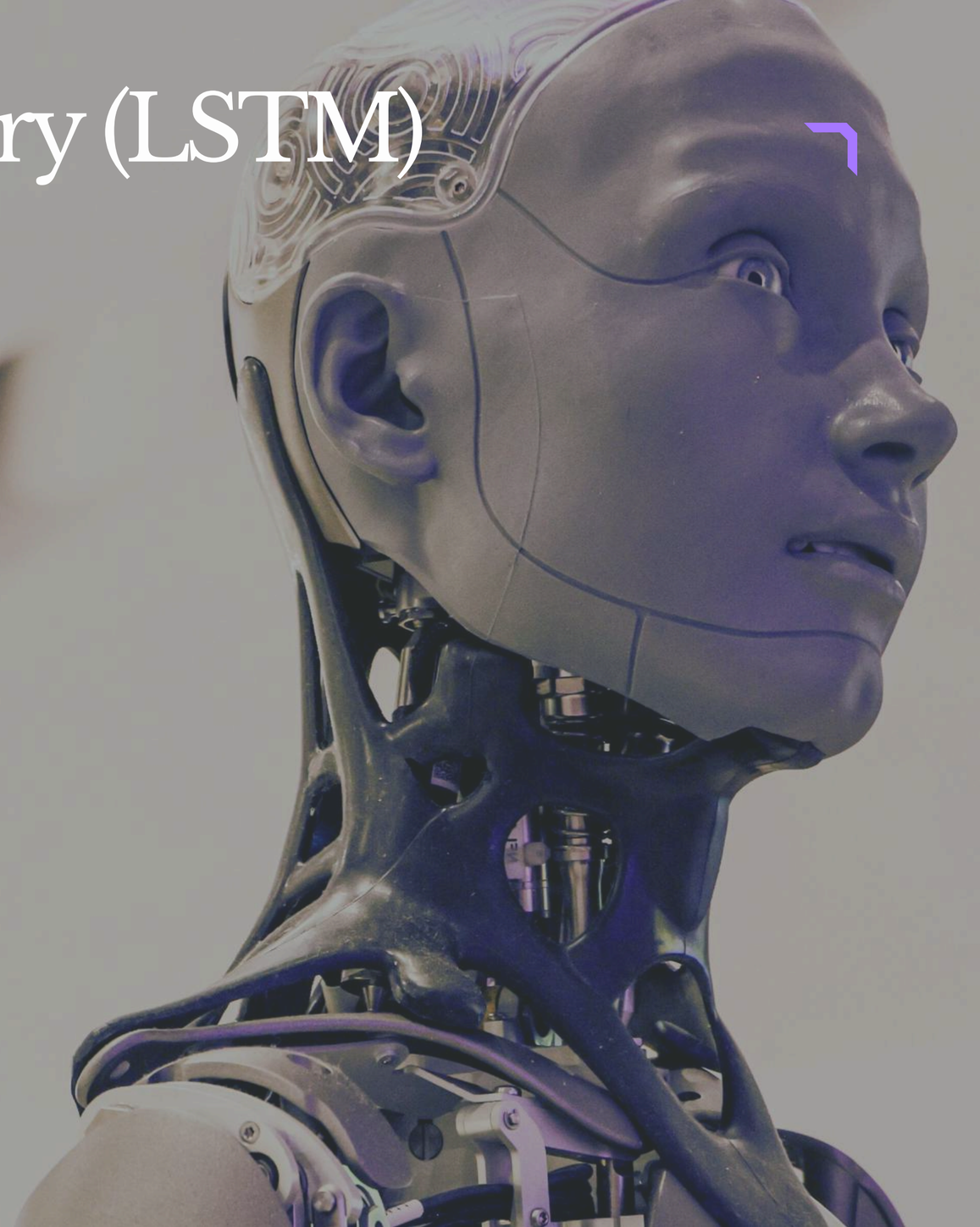
- LSTM is a special type of Recurrent Neural Network (RNN).
- Designed to solve the problem of long-term dependencies.
- Traditional RNNs often forget earlier information in long sequences.
- This happens because of the Vanishing Gradient Problem.
- LSTM overcomes this using memory cells and gates.



Long Short-Term Memory (LSTM)

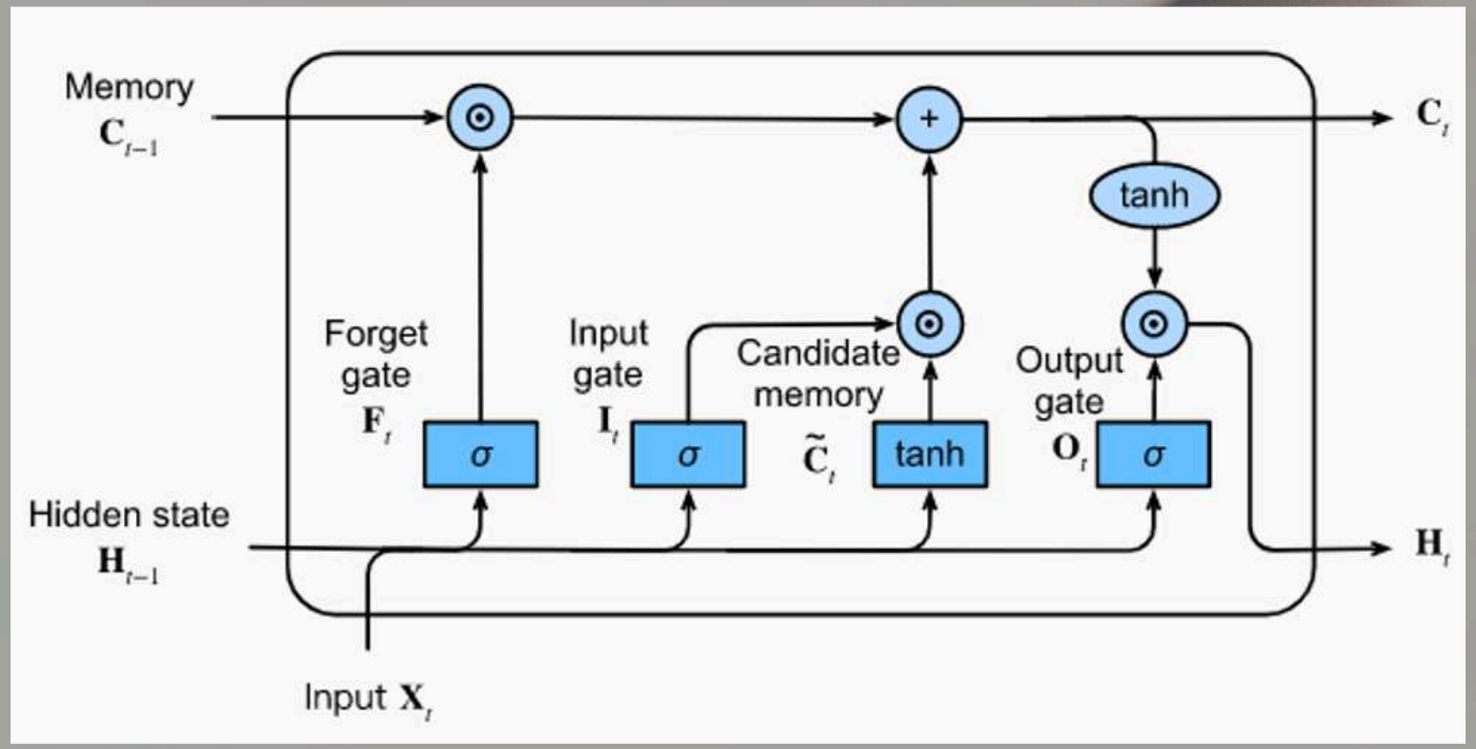
Why LSTM is Needed

- RNNs struggle to remember information for long periods.
- Important past information may get lost during training.
- LSTM can:
- Remember important information
- Forget unnecessary information
- Store information for long durations



Long Short-Term Memory (LSTM)

Architecture



Long Short-Term Memory (LSTM)

LSTM GATES

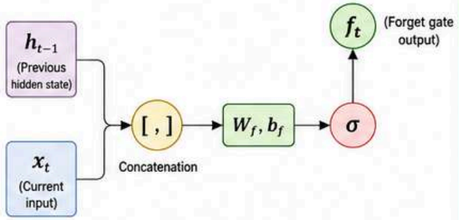
1. FORGET GATE

Decides what information to remove from the cell state.

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

Where,

- f_t = Forget gate output (values between 0 and 1)
- h_{t-1} = Previous hidden state
- x_t = Current input
- W_f, b_f = Weights and bias
- σ = Sigmoid activation function



f_t decides which values from the previous cell state C_{t-1} should be kept or forgotten.

0 → Forget, 1 → Keep

2. INPUT GATE

Decides what new information to add to the cell state.

(a) Sigmoid layer (decides which values to update)

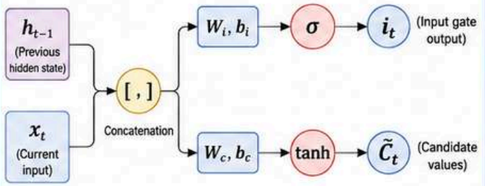
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$

(b) Tanh layer (creates candidate values)

$$\tilde{C}_t = \tanh\left(W_c \cdot [h_{t-1}, x_t] + b_c\right)$$

Where,

- i_t = Input gate output (0 to 1)
- $\tilde{C}_t$ = Candidate values to be added
- W_i, W_c = Weights and bias
- σ = Sigmoid function, tanh = Tanh function



i_t decides which parts to update.

$\tilde{C}_t$ are the new candidate values.

These candidate values are added to the cell state.

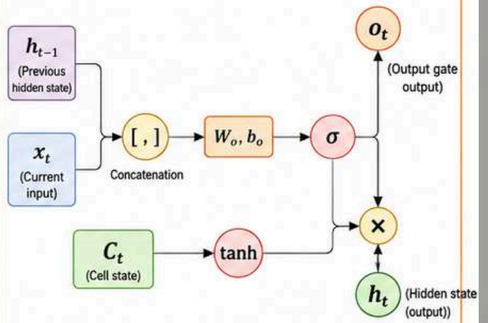
3. OUTPUT GATE

Decides what part of the cell state should be output as hidden state.

$$o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right)$$

Where,

- o_t = Output gate output (0 to 1)
- h_{t-1} = Previous hidden state
- x_t = Current input
- W_o, b_o = Weights and bias
- σ = Sigmoid activation function



o_t decides which part of the cell state C_t will be output as the hidden state h_t.

SUMMARY OF LSTM GATE EQUATIONS

Forget Gate:

$$f_t = \sigma\left(W_f\,[h_{t-1}, x_t] + b_f\right)$$

Input Gate:

$$i_t = \sigma\left(W_i\,[h_{t-1}, x_t] + b_i\right)$$

$$\tilde{C}_t = \tanh\left(W_c\,[h_{t-1}, x_t] + b_c\right)$$

Cell State Update:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate:

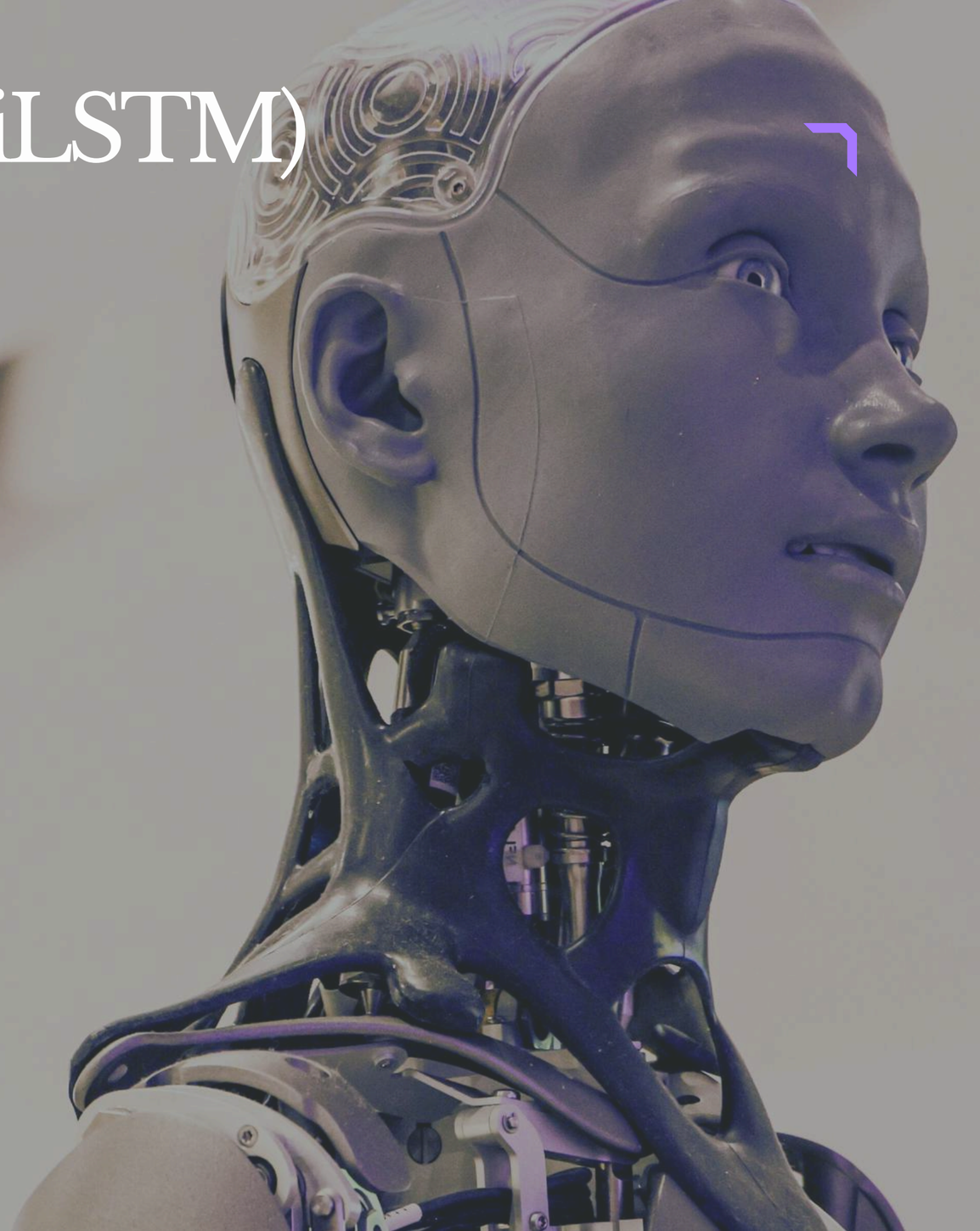
$$o_t = \sigma\left(W_o\,[h_{t-1}, x_t] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Bidirectional LSTM (BiLSTM)

Introduction to BiLSTM

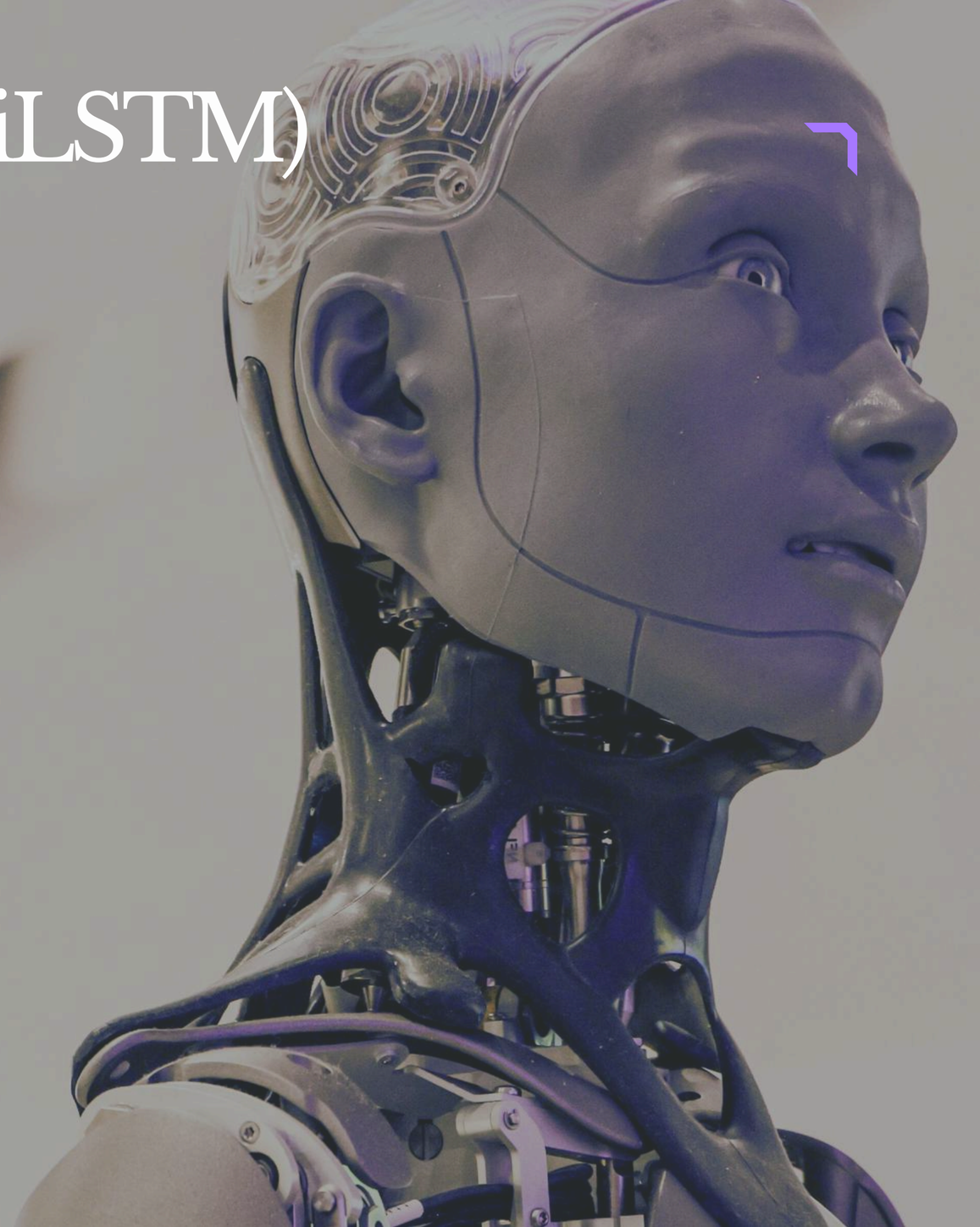
- **Bidirectional LSTM is an advanced type of LSTM.**
- **It processes information in both directions:**
- **Forward direction (past → future)**
- **Backward direction (future → past)**
- **Helps the model understand full context from both sides of a sequence.**



Bidirectional LSTM (BiLSTM)

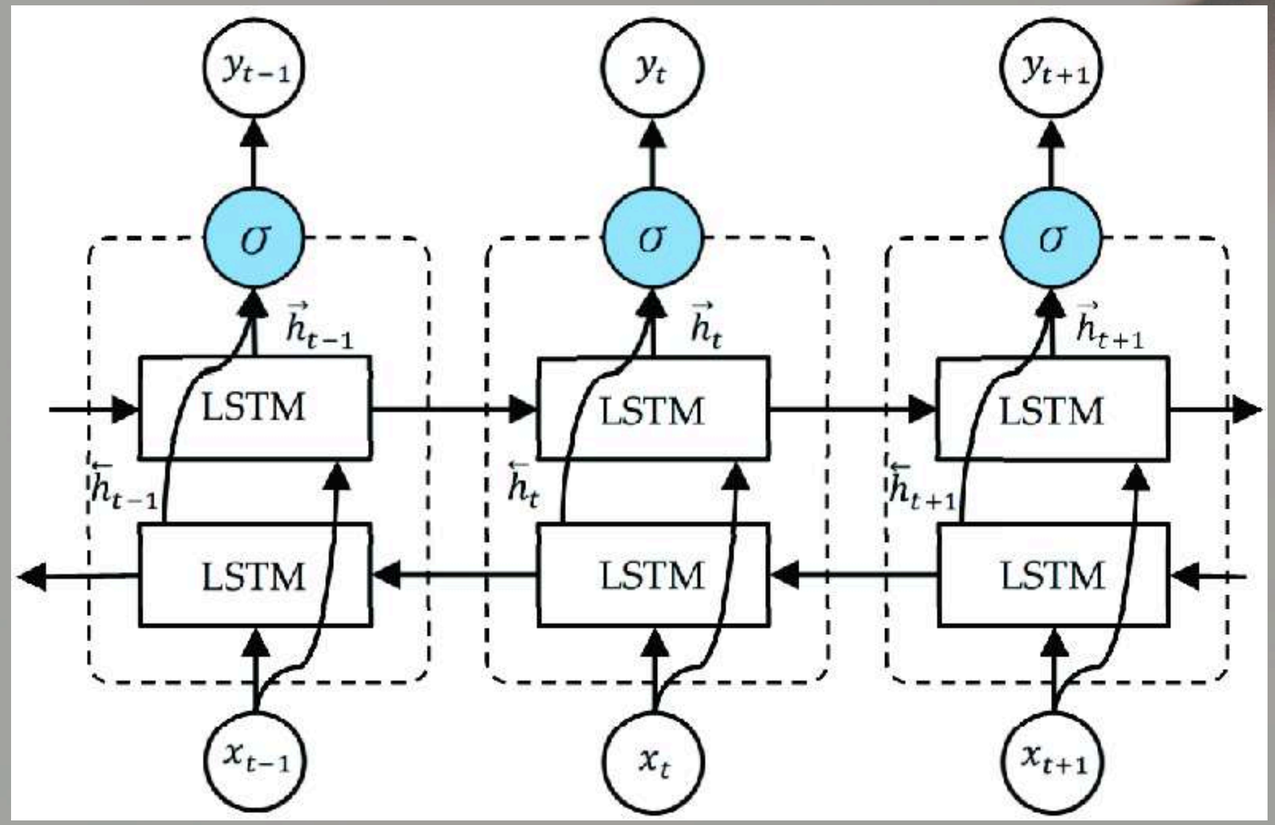
Need for Bidirectional LSTM

- In some tasks, understanding only past information is not enough.
- Future context can also improve predictions.
- BiLSTM captures:
- Previous information
- Upcoming information
- Provides better sequence understanding.



Bidirectional LSTM (BiLSTM)

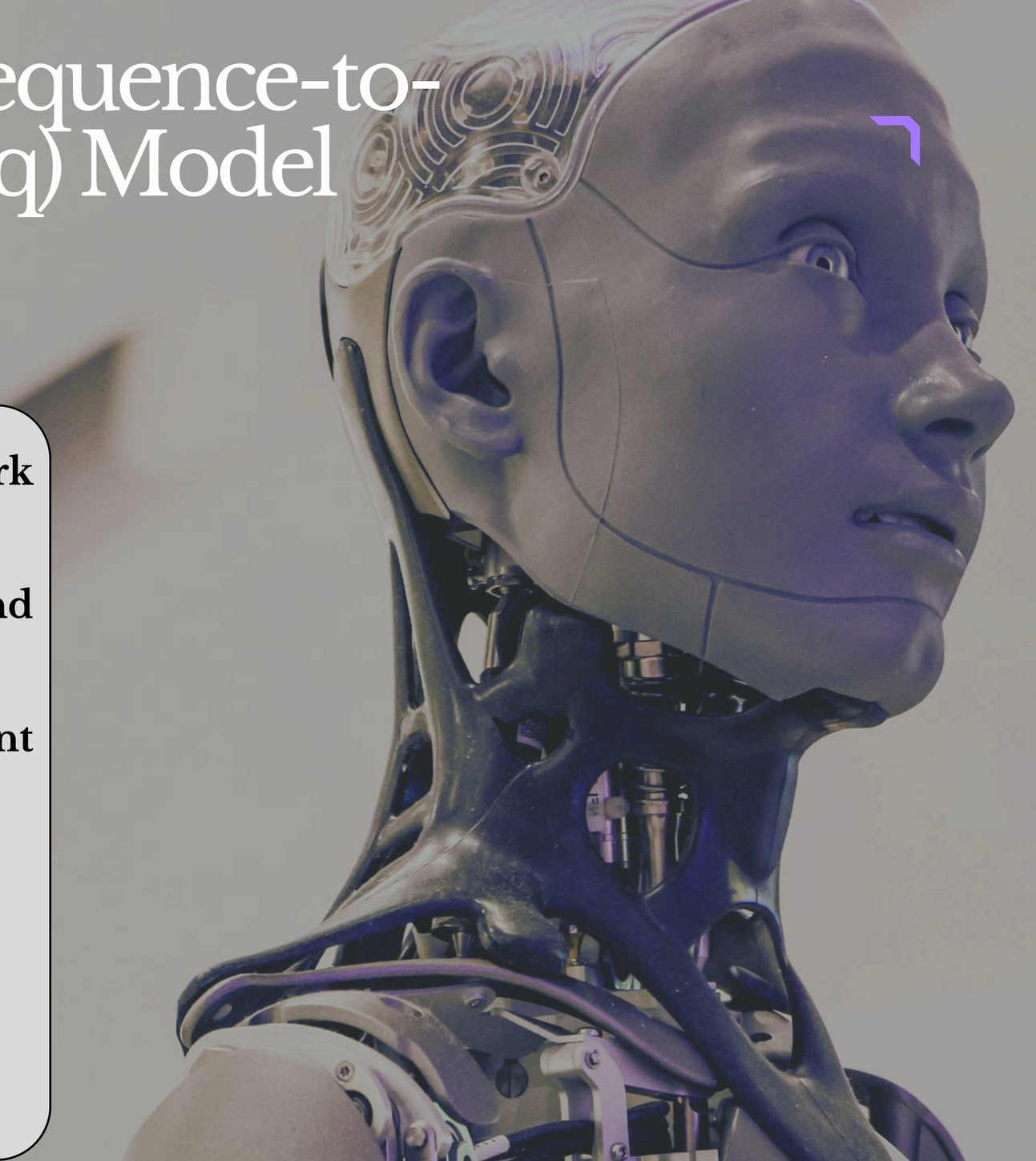
Architecture



Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

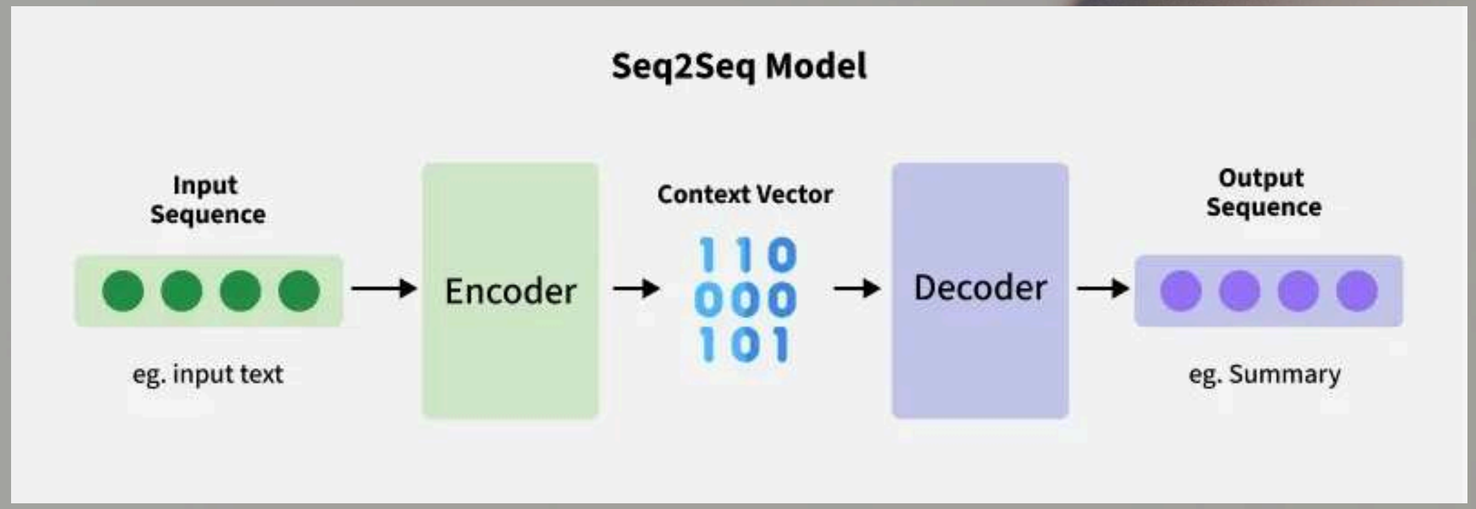
Introduction

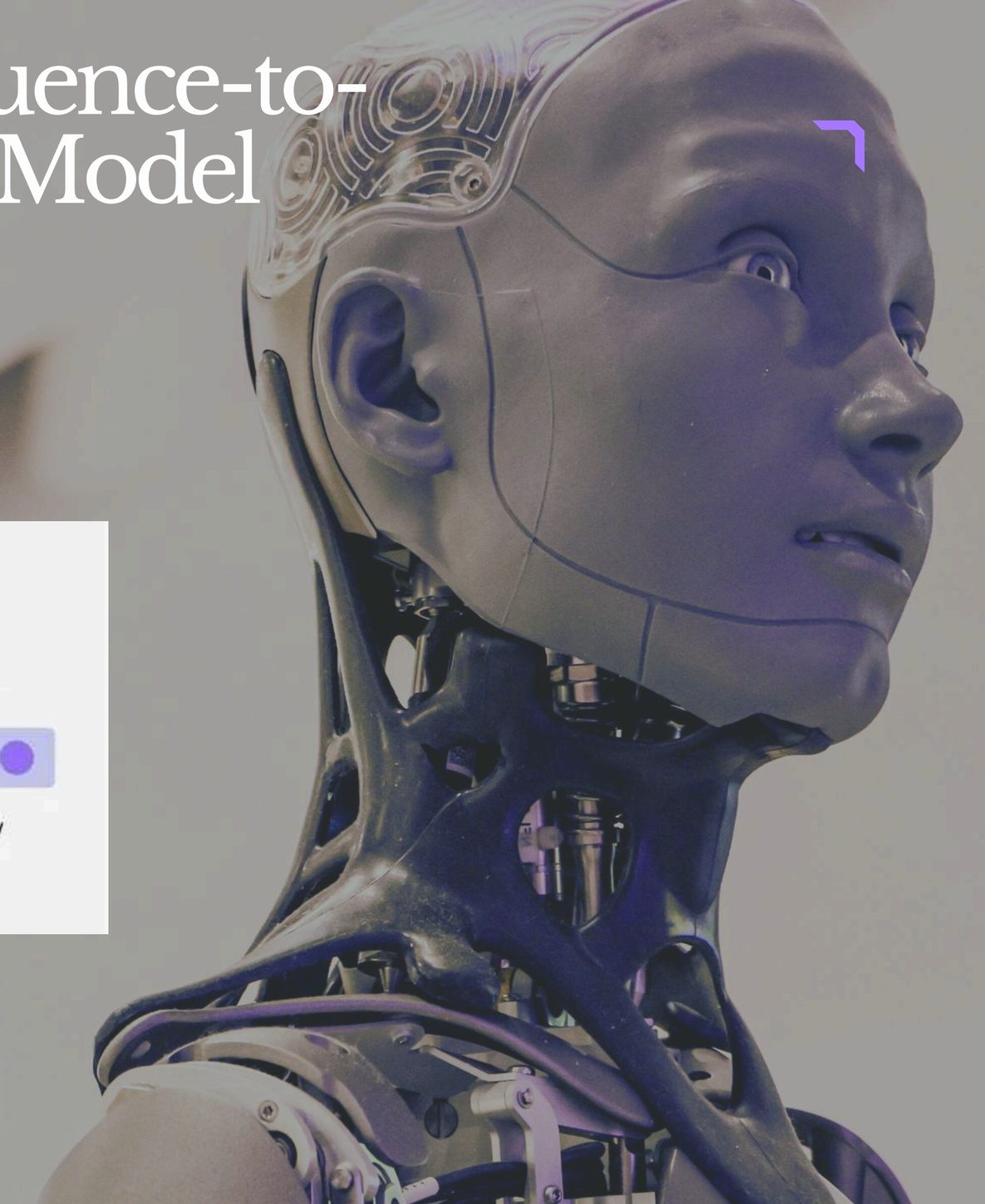
- **Encoder–Decoder is a type of Neural Network architecture.**
- **It is mainly used for tasks where both input and output are sequences.**
- **Input and output sequences can have different lengths.**
- **Commonly used in:**
- **Machine Translation**
- **Text Summarization**
- **Chatbots**



Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

Architecture





Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

1) Encoder

Definition

- **The Encoder reads and understands the input sequence.**
- **It converts the entire input into a meaningful representation called a Context Vector.**

Working of Encoder

- **Usually implemented using:**
- **RNN**
- **LSTM**
- **GRU**

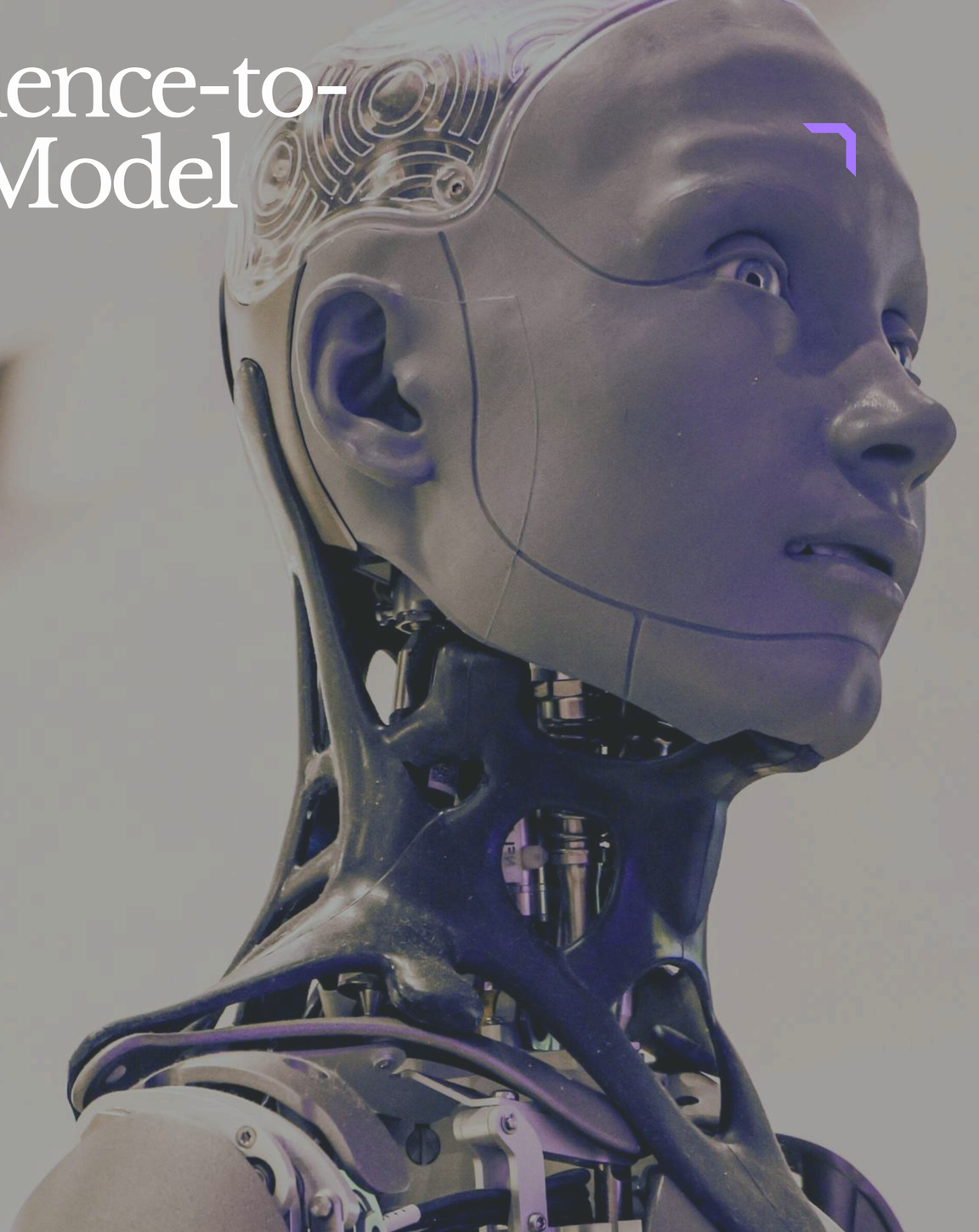
Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

1) Encoder

- **Processes input one word/token at a time.**
- **At every step:**
  - **Updates its hidden state.**
  - **Stores information from previous words.**

Final Output of Encoder

- **After reading the complete sequence, the encoder generates:**
  - **Final Hidden State**
  - **Context Vector**



Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

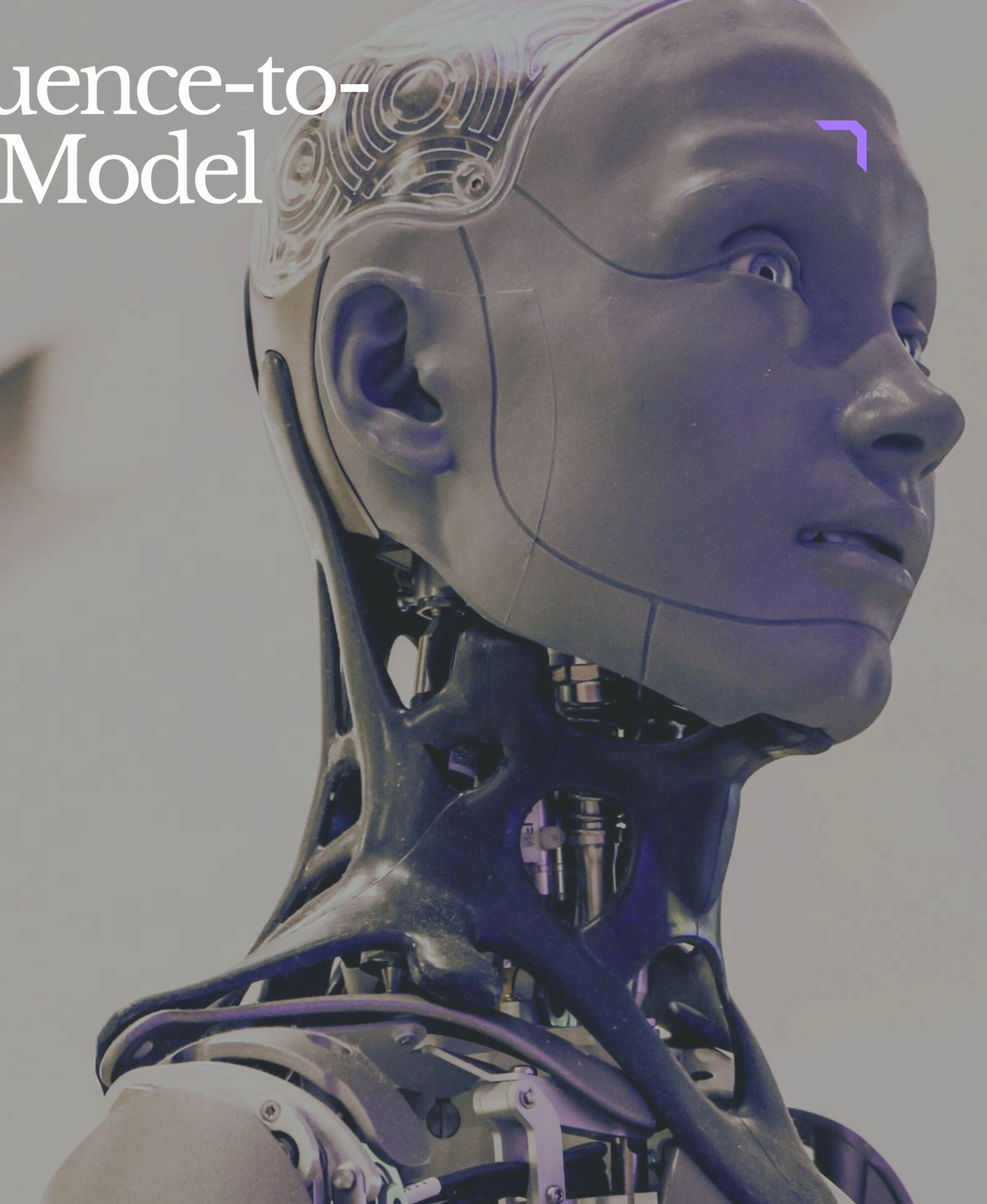
2) Context Vector

Definition

- A fixed-size vector generated by the encoder.
- Contains compressed information about the complete input sequence.

Importance

- Acts as a bridge between Encoder and Decoder.
- Helps the decoder understand the meaning of the input sentence.



Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

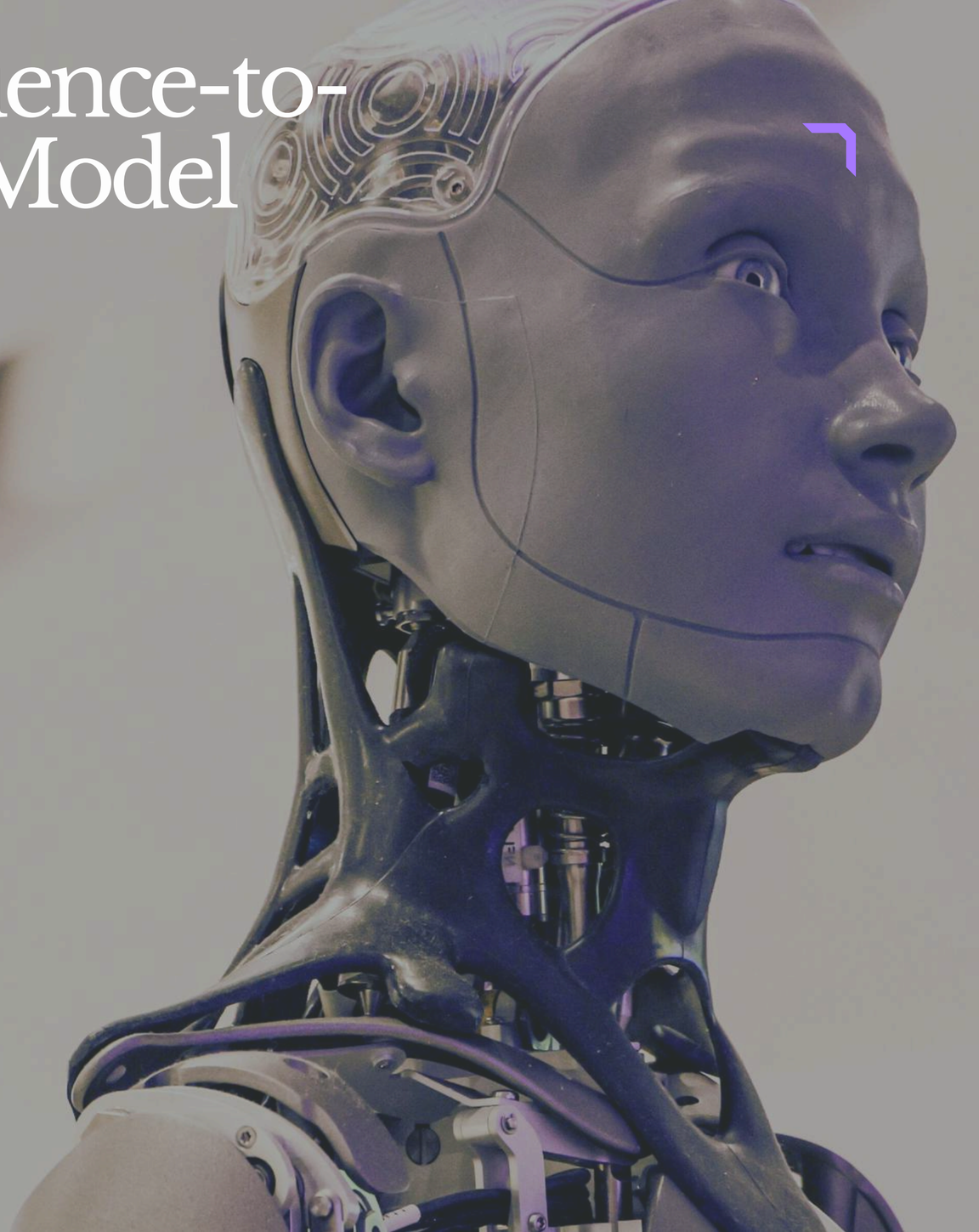
3) Decoder

Definition

- **The Decoder generates the output sequence step-by-step.**

Working of Decoder

- **Usually implemented using:**
  - **RNN**
  - **LSTM**
  - **GRU**
- **Takes Context Vector as input.**
- **Predicts one word at each time step.**



Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

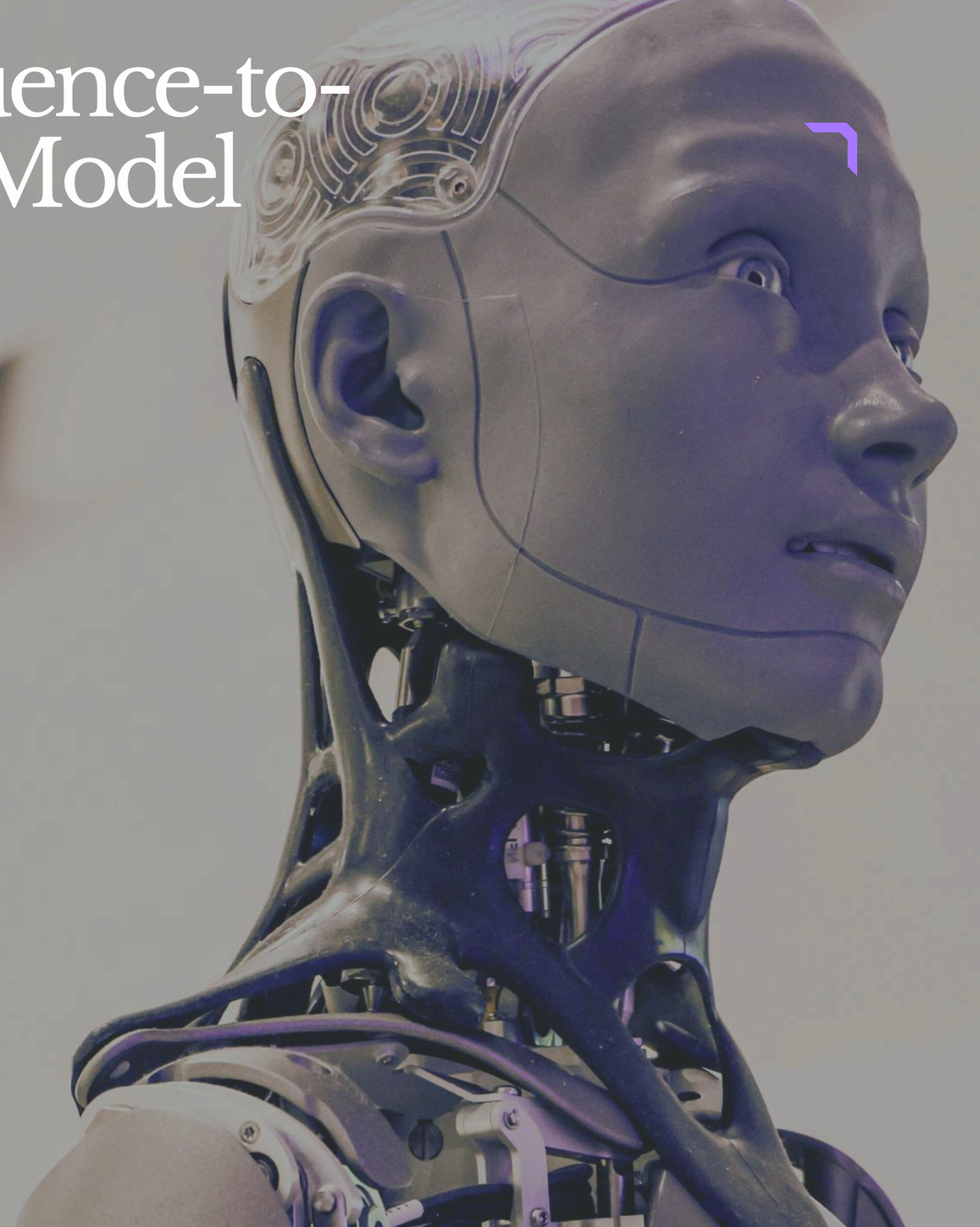
3) Decoder

- **Uses:**
  - **Previous output word**
  - **Previous hidden state**
  - **Context Vector**

Stopping Condition

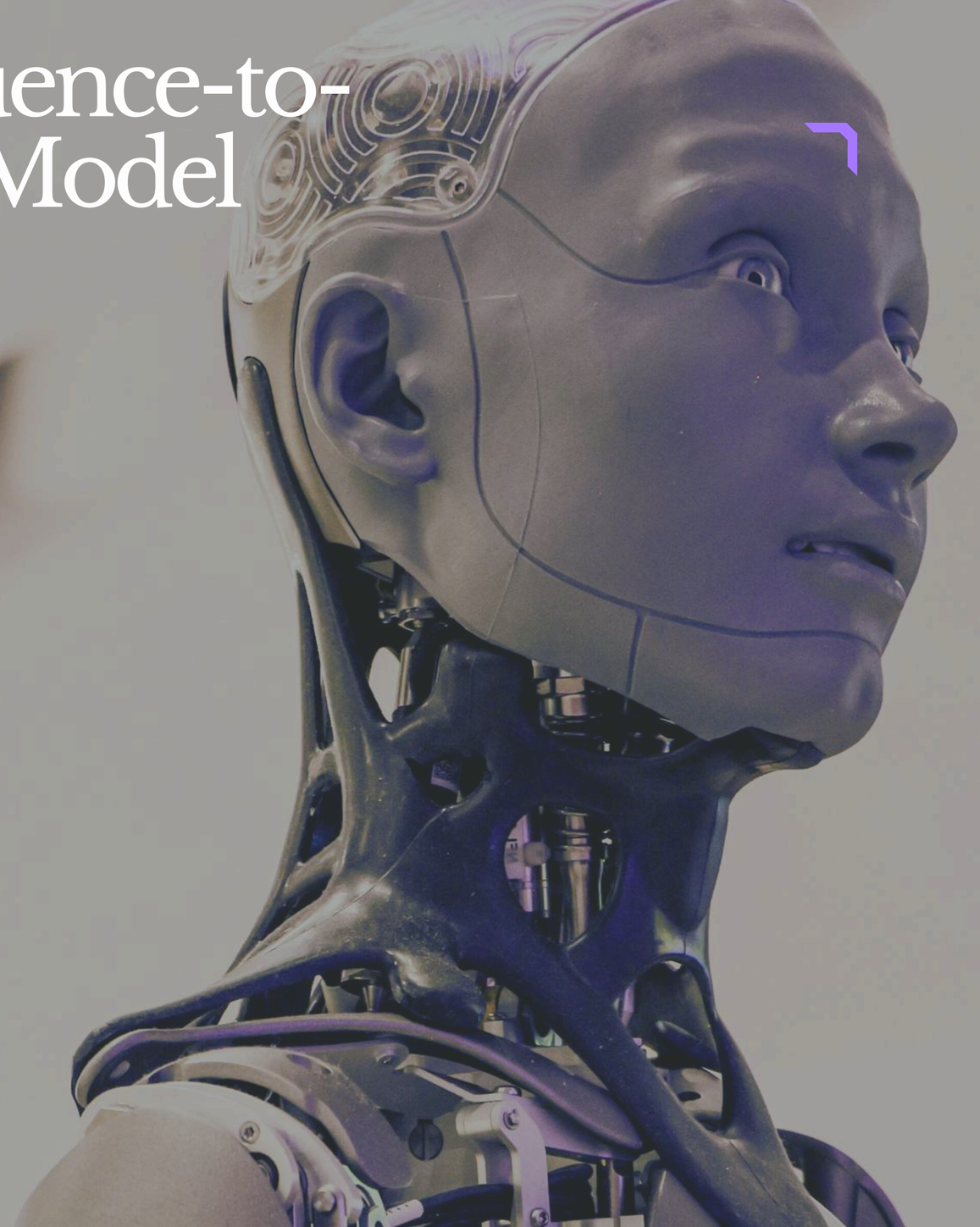
- **Decoder continues generating words until:**
  - **End-of-sequence token (EOS) is reached.**

Features



Encoder–Decoder Sequence-to-Sequence (Seq2Seq) Model

- Handles variable-length sequences.
- Learns sequential dependencies.
- Suitable for language-related tasks.
- Can remember previous information using hidden states.



Default Baseline Models

- Default Baseline Models are simple models created before using complex models like RNN, CNN, or Deep Learning models.
- They act as a starting point or reference for comparing the performance of advanced models.
- Baseline models help us understand whether the complex model is actually improving the results or not.
- These models usually use simple rules or basic algorithms and require very little or no learning.
- Baseline models are easy to implement and fast to evaluate.

Default Baseline Models

- Random Classifier is a baseline model used for classification tasks.
- It randomly guesses the output class without learning from the data.
- It provides a very low benchmark for model performance.
- If a trained model cannot outperform a random classifier, it indicates poor model performance.

- Most Frequent Class Predictor always predicts the most common class present in the training dataset.
- Example: If 80% of emails are “Not Spam,” the model

Default Baseline Models

- This baseline is useful for imbalanced datasets.

- Mean or Median Predictor is commonly used for regression tasks.
- It predicts the mean or median value of the training data for every input.
- Example: In house price prediction, if the average price is 50 lakhs, the model predicts 50 lakhs for all houses.

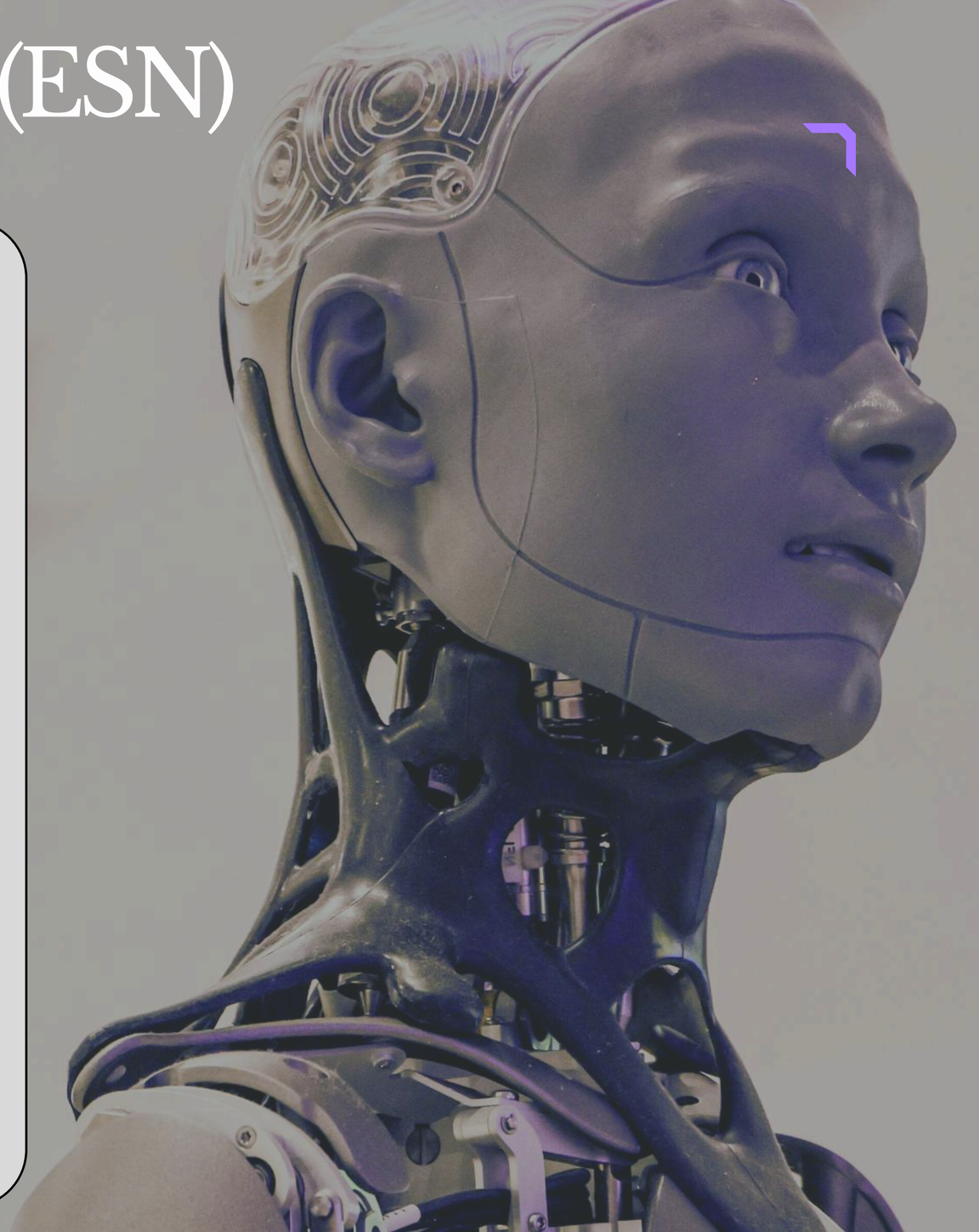
- Rule-Based Models use manually written rules or logic to make predictions

Default Baseline Models

- These models do not learn automatically from data.
- Example: If temperature ⩾ 30°C, predict “Hot.”
- Rule-based models are simple and interpretable.
- Baseline models are important because they provide a benchmark for evaluating machine learning and deep learning models.

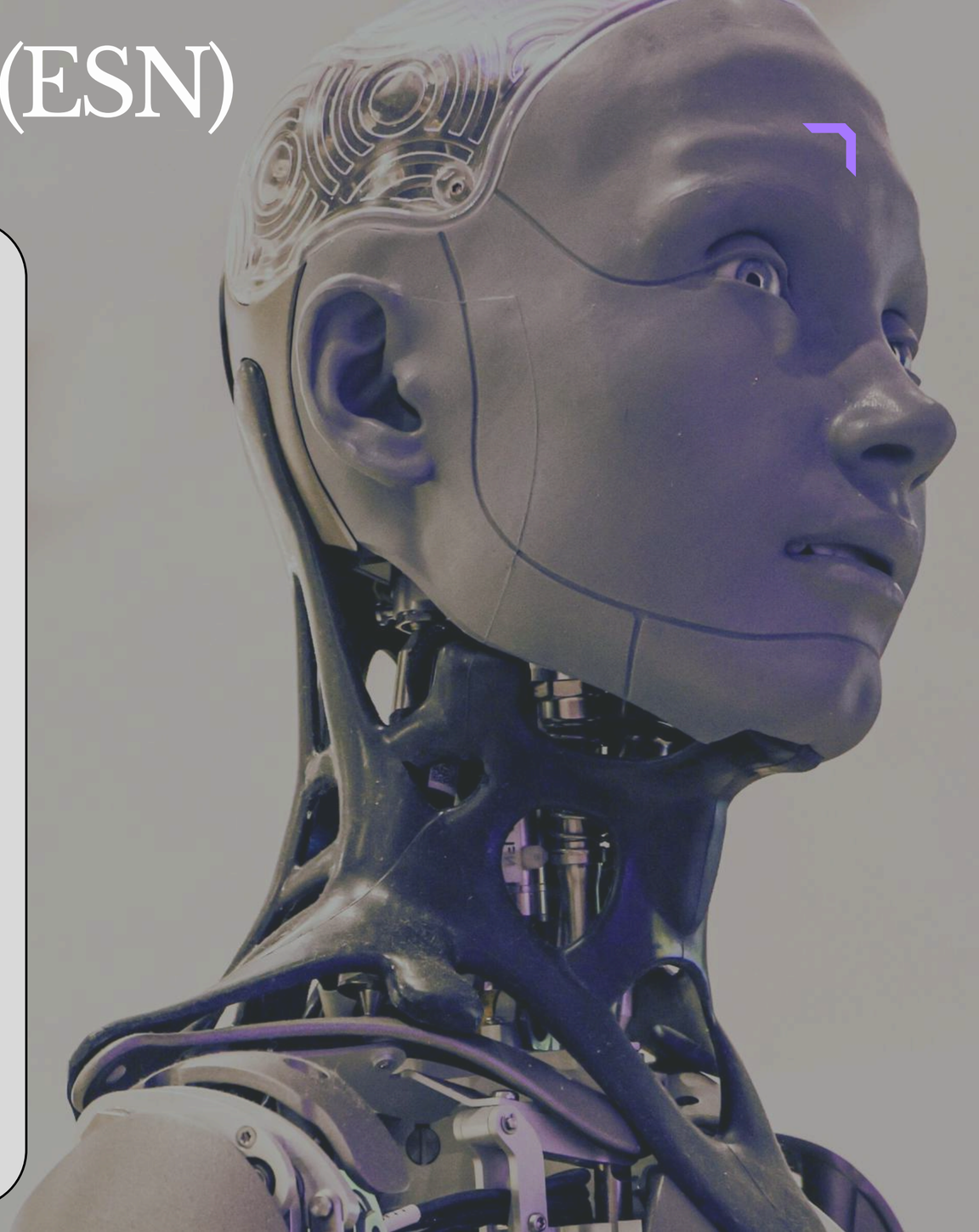
Echo State Networks (ESN)

- Echo State Networks (ESN) are a special type of Recurrent Neural Network (RNN).
- ESNs are designed to simplify training and handle long-term dependency problems in sequence learning tasks.
- Unlike traditional RNNs, ESNs do not train all network weights.
- In ESN, only the output layer is trained while the remaining network stays fixed.
- The fixed internal network is called the Reservoir.
- The Reservoir is a large randomly connected network of neurons.



Echo State Networks (ESN)

- Input data passes through the reservoir and creates complex dynamic patterns called “echoes.”
- These echoes help the network remember information from previous inputs.
- ESNs can capture both short-term and long-term dependencies in sequential data.
- The trained output layer uses the reservoir states to make predictions.
- The reservoir transforms input data into a high-dimensional dynamic representation.
- This transformation helps the network store memory of past inputs for a longer time.



Echo State Networks (ESN)

- ESNs are computationally efficient because only the output layer requires training.
- They avoid complex training methods like Backpropagation Through Time (BPTT).
- ESNs help solve the vanishing gradient problem found in traditional RNNs.
- The internal reservoir weights remain fixed after initialization.
- Training in ESN is faster compared to traditional RNNs and LSTMs.

Leaky Units

- Leaky Units are special types of units used in Recurrent Neural Networks (RNNs).
- They are designed to help the network remember information over different time scales.
- These units are called “leaky” because they slowly update their state instead of completely replacing it at every time step.
- Leaky Units help the network retain past information for a longer duration.
- This makes it easier to learn patterns that occur over both short and long periods of time.
- In a normal RNN, the hidden state is fully updated with each new input.

Leaky Units

- Due to this, traditional RNNs may quickly forget older information.
- Leaky Units solve this problem by combining part of the old state with the new state.
- This allows a small portion of past information to “leak” into the current state.
- At each time step, the new state is calculated as a weighted combination of:
- Previous state
- Current input
- This gradual update helps smooth the changes in memory over time.

Leaky Units

- The Leakiness Factor (α) controls how much old memory is retained.
- The value of α lies between 0 and 1.
- Smaller α values keep more past information.
- Larger α values make the unit respond more quickly to new inputs.
- Formula for Leaky Unit:
- New State=(1−α)×Old State+α×Input
- Here, α represents the leak rate.
- Leaky Units help the network capture patterns occurring at different time intervals.
- They improve long-term memory retention in sequential data.

Bidirectional RNN

Bidirectional RNN (BiRNN)

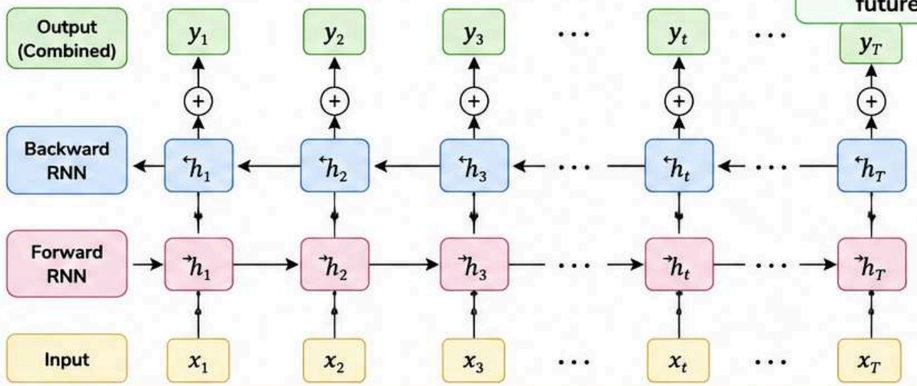
A Bidirectional RNN processes the input sequence in both directions and combines the information from past and future context.

Advantages

- ✓ Better context understanding
- ✓ Improved prediction accuracy
- ✓ More precise output generation
- ✓ Captures both past and future dependencies

How it Works?

- Two separate RNN layers are used.
- One RNN processes the sequence from left to right (Forward).
- The other RNN processes the sequence from right to left (Backward).
- The outputs from both RNNs are combined at each time step.
- This helps the model understand what came before and what comes after each word.



Legend

- x_t : Input at time t
- $\overrightarrow{h}_t$: Forward hidden state
- $\overleftarrow{h}_t$: Backward hidden state
- y_t : Output at time t (Combined)

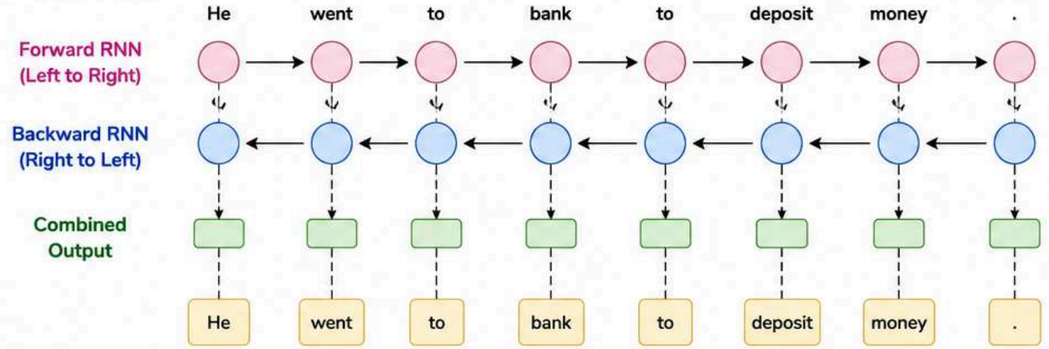
Example

Sentence: **"He went to bank to deposit money."**

- The word "bank" can have multiple meanings:
  - River bank
  - Financial bank
- If the model only looks at previous words, it may be confused.
- By also analyzing future words like "deposit money", the meaning becomes clear.

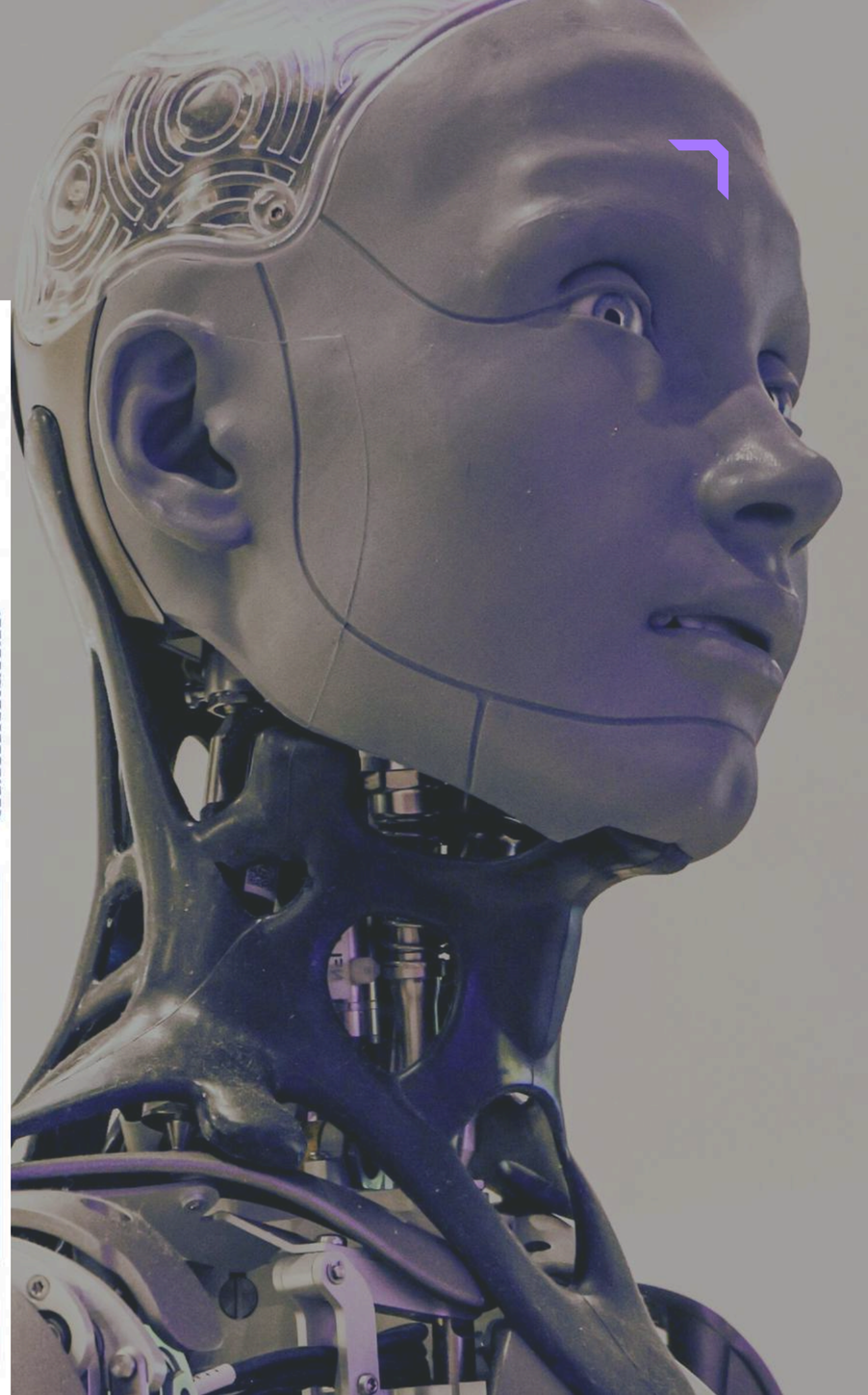
BiRNN uses both left context and right context to make better predictions.

Detailed View



Applications

- NLP
- Speech Recognition
- Text Classification
- Machine Translation
- Time-Series Analysis



Performance metrics of RNN

1. Accuracy

- Accuracy is the percentage of correct predictions made by the model.

Formula

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100$$

Example

- If an RNN correctly predicts 90 words out of 100, accuracy = 90%.
- Higher accuracy indicates better model performance.

2. Loss Function

- Loss function measures how far the model's predictions are from the actual values.
- It helps the model improve by minimizing prediction errors.
- Lower loss values indicate better performance.

Cross Entropy Loss (Classification)

Measures the difference between predicted probabilities and actual class labels.

$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

Mean Squared Error (MSE) (Regression)

Calculates the average squared difference between predicted and actual values.

$$MSE = \frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

3. Perplexity

- Perplexity is mainly used in language modeling tasks.
- It measures how well the RNN predicts a sequence of words.
- Lower perplexity means the model predicts words more accurately.
- A model with low perplexity has better understanding of language patterns.

Formula

$$\text{Perplexity} = 2^{-\frac{1}{N}\sum_{i=1}^{N} \log_2 P(w_i)}$$

4. BLEU Score (for Translation Tasks)

- BLEU (Bilingual Evaluation Understudy) score is used for machine translation tasks.
- It compares the sentence generated by the RNN with a reference sentence.
- It checks translation quality and similarity.
- The BLEU score ranges from 0 to 1.
- Higher BLEU scores indicate better translation quality.

Score Range

0 — 0.5 — 1

Low Quality | Medium Quality | High Quality

5. Confusion Matrix

- Confusion Matrix displays actual vs predicted values in a table format.
- It helps identify where the RNN is making mistakes.
- Useful for analyzing classification performance in detail.
- Helps detect problems like class misclassification or confusion between categories.

Actual Class \ Predicted Class	Positive (1)	Negative (0)
Positive (1)	TP (True Positive)	FN (False Negative)
Negative (0)	FP (False Positive)	TN (True Negative)

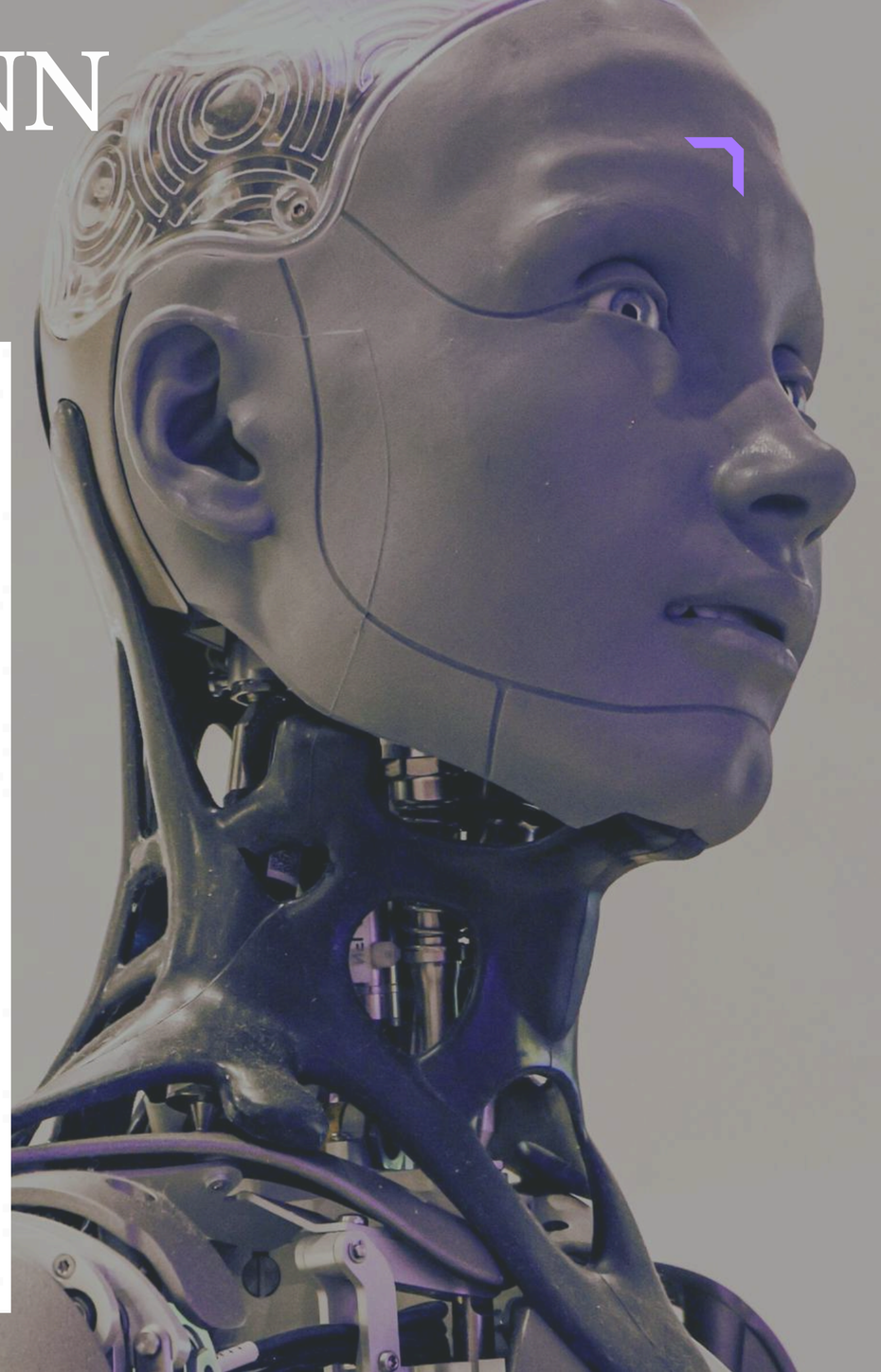
- **TP**: Correctly predicted Positive
- **TN**: Correctly predicted Negative
- **FP**: Incorrectly predicted Positive (False Alarm)
- **FN**: Incorrectly predicted Negative (Miss)

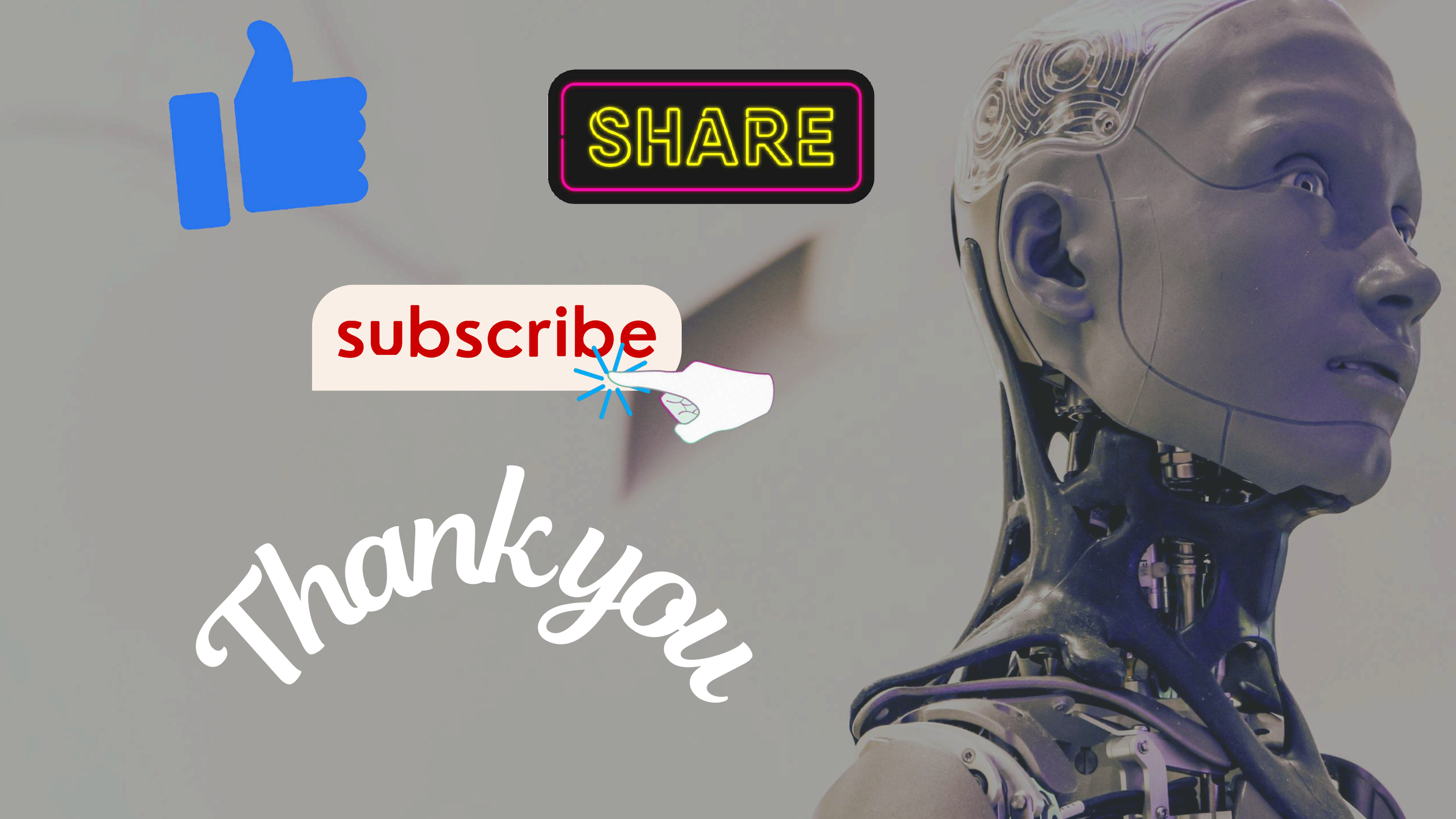
Summary

Accuracy	Measures percentage of correct predictions.
Loss Function	Measures error between predicted and actual values (Cross Entropy for classification, MSE for regression).
Perplexity	Measures how well the model predicts a sequence (lower is better).
BLEU Score	Measures quality of machine translation (range 0 to 1, higher is better).
Confusion Matrix	Shows actual vs predicted values and helps find classification errors.

Why Performance Metrics are Important?

They help compare models, understand strengths & weaknesses, detect errors, and improve overall prediction accuracy.




SHARE
subscribe
Thankyou